

StoresAce Web Services API

Manual Guide

Revisions

Date	Version	Description
2017-11-24	0.1	Initial Version
2018-02-16	1.0	Completed POS Services
2019-07-02	1.1	Added Loyalty Methods
2019-09-25	2.0	Added Head Office Methods
2020-11-21	2.1	Updated Root information with Online Shop
2022-12-07	2.2	New API for Temporary Items
2023-03-10	2.3	Updated information about Cash Drawers
2023-02-05	2.4	New filters and new methods for Sales and Items

Revisions	2
Introduction	3
Authentication	5
API Root - Methods List	5
Access Types	7
GET	7
The Synccounter Argument	9
Access to Single Record	9
Results Format	10
POST	10
HeadOffice API	11
Sales Transactions	12
Purchases Movements	16
Deliveries Movements	19
Stock Movements	22
Inventories	25
Stores	28
Warehouses	29
Families	29
Items	30
Temporary Items	34
Suppliers	36
Customers	38
Employees (POS Salesman)	41
Payment Methods	43
Units	44
POS API	45
Sales Transactions	45
Stores	46
Warehouses	46
Families	47
Items	47
Items & Prices	47
Promotion Menus Items	48
Customers	49
Employees (POS Salesman)	49
Payment Methods	49
Sales Zones (Locals)	50

Cash Drawers (Tills)	51
Cash Drawers Movements (Tender Transactions)	52
Loyalty API	53
Check Available Offers	53
Rescue Offer	56
Customers Purchases API	57
Purchases	57
WebHooks	59
Help & Support	59

Introduction

This document aims to define the WebServices API processes for communications between StoresAce platform and its customers and partners, in order to enable the transmission of information with external systems.

StoresAce provides powerful and flexible Web APIs methods, to easily communicate with external applications.

If you have your own Software or just pretend to integrate your existing applications with the StoresAce BackOffice, you can use all the API features to synchronize all information. It allows you to upload and retrieve all the contents you need.

For example, with the StoresAce API you can get all information about Products and Prices and send back all sales transactions details made on each Point of Sale. At any time, you can retrieve what new products are ready to send to stores, or what prices need to be changed.

All the features are available through [REST Web Services](#) in JSON format. This means that, for any available entity feature, you can View, Retrieve, Create or Update any existing field. A Web Browsable API is also available to navigate over all features.

Authentication

All calls to Web API methods are made through a secure authentication system.

You need to ask for your own API Username and Password to validate each connection. Each environment has their own API user and password. Traditional Web users don't have access to API methods. If you don't have one, please contact our support team and request a new API User.

Depending on the programming language you might be using, you will need to configure the specific URL you want to call, passing the correct username and password to it. Bellow is an example API Request on Products List Method, using the command line [HTTPIe](#) client for JSON support:

http -a user:password GET https://[environment].storesace.com/dimas/api/pos/items/

API Root - Methods List

The StoresAce API has a Root Web Browsable access from where all methods are displayed. This means you can view all available features, and navigate through them using your normal Web Browser. Simple access the API main root link, on your environment, to check all available methods:

[https://\[environment\].storesace.com/dimas/api/](https://[environment].storesace.com/dimas/api/)

This link will present the StoresAce API Root Page, listing all available methods to communicate with external applications. If you need some new specific method not listed on your environment, please contact our support team.

Api Root

Api Root

Base Root for all API Methods

:param request:

:param format:

:return:

GET /dimas/api/

HTTP 200 OK

Allow: GET, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "POS Methods": "https://demo.storesace.com/dimas/api/pos/",
  "HeadOffice Methods": "https://demo.storesace.com/dimas/api/headoffice/",
  "Loyalty Methods": "https://demo.storesace.com/dimas/api/loyalty/",
  "SyncCounter Methods": "https://demo.storesace.com/dimas/api/synccounter/",
  "Synchronizer Methods": "https://demo.storesace.com/dimas/api/synchronizer/",
  "Devices Methods": "https://demo.storesace.com/dimas/api/devices/",
  "Customers Purchases": "https://demo.storesace.com/dimas/api/customerspurchases"
}
```

The Main API Root is the access portal to all available methods.

Mainly, the methods are organized in two main Areas: POS Methods and HeadOffice Methods. These represent the services used to integrate with POS Systems and with ERP (Headoffice)

Lisboa
Av. Eng. Arantes e Oliveira, 3 R/C, Lisboa, Portugal

Porto / Head Office
Rua Fernando Namora 117, Porto, Portugal

info@storesace.com

For the ones with multi-stores
Anyplace, anytime and anywhere.

Level up!
storesace.com

systems, and that's why the information may vary depending on which system you are communicating with.

Other roots are also available for integration with third party systems, such as loyalty programs, IoT Devices, Ecommerce WebSites or Online Shops.

Each method details and content are accessible by it's own endpoint url listed in this Root Page. Simple click on each method link url to access all instance records with all fields and properties. You will be also able to navigate on all records list, and make GET or POST events on specific records to retrieve or update some particular content.

Api Root / Api Pos Root / Items List

Items List

OPTIONS GET ▾

« 1 2 3 ... 46 »

GET /dimas/api/pos/items/?page=2

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 1129,
  "next": "https://demo.storesace.com/dimas/api/pos/items/?page=3",
  "previous": "https://demo.storesace.com/dimas/api/pos/items/",
  "results": [
    {
      "Id": 10018,
      "ItemId": "505063",
      "Status": 1,
      "Type": 3,
      "TypeDescription": "Purchase & Sale",
      "ItemDescription": "Artigo 505063 (CV)",
      "ItemPublicDescription": "Artigo 505063 (CV)",
      "ItemShortDescription": "Artigo 505063 (CV)",
      "ItemShortDescription2": "Artigo 505063 (CV)",
      "ItemLongDescription": null,
      "ExternalCode": null,
      "ParentFamilyId": 2000,
      "ParentFamilyLevelId": 2,
      "ParentFamilyDescription": "COMIDAS",
    }
  ]
}
```


Access Types

There are two access modes you call the services described: GET and POST.

GET

The GET method is used to view and retrieve individual or large records list. You can access via Web Browser navigation, or by API method call using the GET request type. Bellow is a list of some arguments that can be passed to all methods:

Argument	Description
format	Display Format Type (Default: Web HTML Browsable). Pass json to return content in JSON format, or xml to return in XML format.
page_size	Number of results to be returned into each page. Note that some API methods limit the minimum and maximum count of records.
page	Pagination index to display the records list. Full result sets are split into individual pages of data, based on page_size.
store	Store Code to filter only results available on a specific store. Example: Retrieve only Products available to sell on Store with code 1.
synccounter	<u>Very Important</u> : Filter records that were created or updated since your last call. The synccounter object represents an EPOCH and is a long integer value that matches current date and time. To avoid retrieving and updating the entire record set, save your last returned synccounter and easily retrieve only changes occurred after your last synchronization.

The Synccounter Argument

In all of these methods there is a parameter that can be passed, called synccounter. This is an *EPOCH*, and allows you to obtain lists of entities/movements that have been created or changed since your last synchronization.

For example, to obtain the entire list of items, you can pass `synccounter=-1` and save the current epoch or check the max of the `synccounter` attribute of each article in this list, which for instance would be 150000. In the next call, pass this value in the parameter to obtain the list only of those that have changed or been created since the previous call, as follows:

`https://[environment].storesace.com/dimas/api/pos/items/?synccounter=150000`

Note: It is a requirement that you make calls to any API method always passing this argument. The same applies to the entire movements API.

Access to Single Record

In all Services, it is possible to directly access a single record, by using StoresAce Unique Internal Integer Identifier (Id), which is also returned in all services.

To do so, just append the Id number to each service URL. For instance, if you want to access the Item with Id 1234, call the following URL:

`https://[environment].storesace.com/dimas/api/pos/items/1234/`

Results Format

The result, when in JSON format, will be returned like the following syntax:

```
{
  "Count":           // total full list records count
  "next":            // Method next results page link
  "previous":        // Method previous results page link
  "results": [       // START of result array list
    {                // record 1
      "fieldnameA":  // fieldvalue
      "fieldnameB":  // fieldvalue
      ...
    },
    {                // record 2
      "fieldnameA":  // fieldvalue
      "fieldnameB":  // fieldvalue
      ...
    }
  ]
}
```

```

},
...
{
    "fieldnameA": // record n (defined by page_size argument)
                  // fieldvalue
    "fieldnameB": // fieldvalue
    ...
},
]
// END of result array list
}

```

POST

The POST request type is used to both create or update existing records. All the API methods are prepared to update records if they exist, or create new ones when they are not found.

It can also create and update multiple records in a single request, passing to the JSON format an array list of elements, just like the GET request type returns an array for all records.

The Web Browsable API has a built-in POST request feature using JSON format, like bellow:

Media type:

Content:

```

{
  "InvoiceNo": "FS 10",
  "TillId": "0",
  "WorkstationId": 1,
  "InvoiceStatus": 0,
  "InvoiceTime": "2018-01-01T12:52:43",
  "TotalNetAmount": 2.40,
  "CustomerId": null,
  "TotalTransactionAmount": 2.95,
  "SalesmanId": 821,
}

```

In case of successfully update or create, a `{"status": "1"}` message json is returned, or the full created/updated record json details is returned.

If the information posted has some validation errors, it will output all errors found in each field:

```
{  
  "InvoiceDate": [  
    "This field may not be null."  
  ],  
  "StoreId": [  
    "This field may not be null."  
  ]  
}
```

HeadOffice API

The HeadOffice Root contains all methods used to integrate with any ERP or Accountant External System. It allows to retrieve mainly the resume of all Sales, Purchases and Stocks for all stores defined in StoresAce.

As it is the aim of an ERP, it doesn't need to have the entire full details of all movements. For this, StoresAce has the capacity to output only the resumed or grouped information, per store, per day or even per document, to easily integrate only the totals values that an ERP needs for Financial or Accounting purposes.

The HeadOffice API Root is available at:

[https://\[environment\].storesace.com/dimas/api/headoffice/](https://[environment].storesace.com/dimas/api/headoffice/)

```
"Stores": "https://demo.storesace.com/dimas/api/headoffice/stores/?format=json&page_size=10",
"Warehouses": "https://demo.storesace.com/dimas/api/headoffice/warehouses/?format=json&page_size=10",
"Families": "https://demo.storesace.com/dimas/api/headoffice/families/?format=json&page_size=10",
"Groups": "https://demo.storesace.com/dimas/api/headoffice/groups/?format=json&page_size=10",
"Items": "https://demo.storesace.com/dimas/api/headoffice/items/?format=json&page_size=10",
"Suppliers": "https://demo.storesace.com/dimas/api/headoffice/suppliers/?format=json&page_size=10",
"Customers": "https://demo.storesace.com/dimas/api/headoffice/customers/?format=json&page_size=10",
"Salesmans": "https://demo.storesace.com/dimas/api/headoffice/salesmans/?format=json&page_size=10",
"Units": "https://demo.storesace.com/dimas/api/headoffice/units/?format=json&page_size=10",
"Payment Types": "https://demo.storesace.com/dimas/api/headoffice/paymenttypes/?format=json&page_size=10",
"Purchases [Full]": "https://demo.storesace.com/dimas/api/headoffice/purchases/full/?format=json&page_size=10",
"Purchases [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/purchases/?format=json&page_size=10",
"Accounting Differences [Full]": "https://demo.storesace.com/dimas/api/headoffice/settlements/?format=json&page_size=10",
"Settlements Notes [Full]": "https://demo.storesace.com/dimas/api/headoffice/settlementnotes/?format=json&page_size=10",
"Warehouse Transfers [Full]": "https://demo.storesace.com/dimas/api/headoffice/warehousetransfers/full/?format=json&page_size=10",
"Warehouse Transfers [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/warehousetransfers/?format=json&page_size=10",
"Waste Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/wastemovements/full/?format=json&page_size=10",
"Waste Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/wastemovements/?format=json&page_size=10",
"Entry Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/entrymovements/full/?format=json&page_size=10",
"Entry Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/entrymovements/?format=json&page_size=10",
"Exit Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/exitmovements/full/?format=json&page_size=10",
"Exit Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/exitmovements/?format=json&page_size=10",
"Production Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/productionmovements/full/?format=json&page_size=10",
"Production Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/productionmovements/?format=json&page_size=10",
"Inventories [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/inventory/?format=json&page_size=10",
"Sales [Full]": "https://demo.storesace.com/dimas/api/headoffice/sales/?format=json&page_size=10",
"Sales [Store/Date]": "https://demo.storesace.com/dimas/api/headoffice/sales/resume/?format=json",
"Sales [Store/Date/DocType]": "https://demo.storesace.com/dimas/api/headoffice/sales/resume/dt/?format=json",
"Sales [Store/Date/TaxNumber]": "https://demo.storesace.com/dimas/api/headoffice/sales/resume/tn/?format=json",
"Sales [Store/Date/DocType/TaxNumber]": "https://demo.storesace.com/dimas/api/headoffice/sales/resume/dttn/?format=json"
```

Sales Transactions

With this Web Service you can access the full list of Sales Transactions, including all details, or a resumed List with Totals, grouped by Stores, Days, Document Types and/or Customers.

It contains full Invoices Details, like nature, total price, tax values, customer identification, terminal, cash drawer and sale zone, date and time, employee, products quantities and prices details and payment tenders and amounts.

There are 5 API links for Sales Information. Choose the correct one depending on which level of detail/grouped information you pretend to get:

```
"Sales [StoreResume]": "http://burgerking.localhost:8000/dimas/api/headoffice/export/sales/resumed/?format=json&store=1&date=2024-03-02",
"Sales [StoreDetails]": "http://burgerking.localhost:8000/dimas/api/headoffice/export/sales/details/?format=json&store=1&date=2024-03-02",
"Sales [Store/Date]": "http://burgerking.localhost:8000/dimas/api/headoffice/sales/resume/?format=json&date=2024-03-02",
"Sales [Store/Date/DocType]": "http://burgerking.localhost:8000/dimas/api/headoffice/sales/resume/dt/?format=json&date=2024-03-02",
"Sales [Store/Date/TaxNumber]": "http://burgerking.localhost:8000/dimas/api/headoffice/sales/resume/tn/?format=json&date=2024-03-02",
"Sales [Store/Date/DocType/TaxNumber]": "http://burgerking.localhost:8000/dimas/api/headoffice/sales/resume/dttn/?format=json&date=2024-03-02",
"Sales [Full]": "http://burgerking.localhost:8000/dimas/api/headoffice/sales/?format=json&page_size=10&date=2024-03-02",
```

Note: For any Grouped method, the grouping is made by Item Accounting Code that can be filled from each Item Page. It means that you can set your External Accounting ERP Codes in StoresAce Items and retrieve values grouped by each Code.

In case Items Accounting codes are not filled, the Grouped Methods return a default group having the Item' Classification and Vat Details.

Sales [StoreResume]	Resumed Total Sales per Store and Date, with Totals by Tenders, VATs and Zones.
Sales [StoreDetails]	Detailed Sales at Transaction Level, with essential information for HeadOffice Integration. Full list of sales per store and date is returned.
Sales [Store/Date]	Transactions Totals with details grouped by Store and Date. The grouping is made by Item Accounting Code that can be filled from each Item Page. It means that you can set your External Accounting ERP Codes in StoresAce Items and retrieve values grouped by each Code.
Sales [Store/Date/DocType]	Transactions Totals grouped by Store, Date and Document Type
Sales [Store/Date/TaxNumber]	Transactions Totals grouped by Store, Date and Customer Tax Number

Sales [Store/Date/DocType/TaxNumber]	Transactions Totals grouped by Store, Date, Document Type and Customer Tax Number
Sales [Full]	Full Transactions with all Details

Important Considerations:

- For the Grouped Information, the StoresAce Accounting Module is required.
- Everytime a Sale transaction is modified, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Sales Methods to filter results:

Argument	Type	Details
TaxRegistrationNumber	Text	Company Tax Number to filter Sales from all Company' Stores
store	Text	Store Code to filter Sales from a particular Store
accounting_store	Text	Store Accounting Code to filter Sales from a particular Store
date *	Date (YYYY-MM-DD)	Filter Sales Transactions in a particular Day. <i>* <u>mandatory field</u></i>
InvoiceNo	Text	Filter any specific document by its Number (Invoice Number)
InvoiceSearch	Text	Filter any documents containing search text into Invoice Number
VoucherNo	Text	Filter Sales Transactions associated with a Voucher (Voucher Number)
RedeemStore	Text	The Store Code, to filter Sales with Vouchers available to redeem on this store.

The Output in Json format for the Full Details is as follows:

```
[{
  "Id": 12745,
  "StoreId": "1",
  "InvoiceNo": "FS 133/58728",
  "InvoiceDate": "2016-03-16T13:26",
  "InvoiceTime": "2016-03-16T13:26",
  "InvoiceStatus": 0,
  "Zoneld": 4,
  /*StoresAce Unique Integer Identifier*/
  /*StoresAce Store Code*/
  /*Document Number (Invoice, Credit Note, etc)*/
  /*Transaction Date and Time*/
  /*Transaction Date and Time*/
  /*1 if Cancelled, or 0 if Normal*/
  /*StoresAce Sale Zone Code*/
}]
```

```

"WorkstationId": 4,
"TillId": 4,
"SelfExpenditure": 0,
"TransDocument": "FT",
"TransNature": "FT",
"TransDocumentDescription": "Fatura",
"SalesmanId": 10,
"CustomerId": null,
"CustomerCompanyName": null,
"CustomerTaxId": null,
"TotalNetAmount": null,
"TotalTaxAmount": null,
"TotalTransactionAmount": 9.59,
"TotalLineItemDiscountAmount": 0.0,
"TotalPaidAmount": 0.0,
"CustomerCount": 1,
"Synccounter": 0,
"SaleTransactionDetails": [
{
  "ItemId": 521792,
  "ParentItemId": "521792",
  "ParentLevel": 1,
  "LineNumber": 1,
  "ProductDescription": "Artigo A",
  "Description": "Artigo A",
  "Quantity": 1.0,
  "CancelRecord": 0,
  "WasteRecord": 0,
  "TaxIncludedPrice": 0.0,
  "TotalLineItemDiscountAmount": 0.0,
  "TaxPercentage": 10.0,
  "SalesmanId": 10,
  "TotalTaxIncludedAmount": 0.0,
  "TotalNetBaseTaxAmount": 0.0
}],
"TenderlineItems": [
{
  "PaymentAmount": 9.59,
  "TenderId": 11
}
]

```

```

/*StoresAce Terminal Code*/
/*StoresAce Till Code*/
/* 1 if it's a Self Employee Consumption, 0 if not*/
/*Document Code*/
/*Document Nature*/
/*Document Description*/
/*StoresAce Employee Code*/
/*StoresAce Customer Code*/
/*Customer Name*/
/*Tax Number*/
/*Liquid Total without Vat*/
/*Vat Total*/
/*Final Total with Vat*/
/*Discount Total*/
/*Total Paid*/
/*Number of Persons in Sale*/
/*Transaction EPOCH Synccounter*/
/*Array with all Sale Details*/

/*StoresAce Item Code*/
/*Parent Menu Item Code*/
/*Menu Level Index if ParentItemId is defined*/
/*Document Line Number*/
/*Product Description*/
/*Product Description*/
/*Detail Quantity (Float)*/
/* 1 if it's a Cancelled Detail*/
/* 1 if it's a Waste Cancelled Detail*/
/*Unit Price with Vat*/
/*Total Line Discount*/
/*Vat Tax Percent*/
/*Employee Code responsible for the Detail*/
/*Line Final Total Value with Vat*/
/*Line Total Value without Vat*/

/*Array with all Sale Payment Types*/

/*Value*/
/*StoresAce Payment Type Code*/

```


Purchases Movements

With this Web Service you can access the full list of Purchases Invoices and Credits Movements, including all details, or a resumed List where all details are grouped into totals for each Document Movement.

It contains full Purchases Movements Details, like nature, total price, tax values, supplier identification, date and time, products quantities and purchased prices details.

There are 2 API links for Purchases Information. Choose the correct one depending on which level of detail/grouped information you pretend to get:

```
"Purchases [Full]": "https://demo.storesace.com/dimas/api/headoffice/purchases/full/?format=json&page_size=10",
"Purchases [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/purchases/?format=json&page_size=10",
```

Note: For any Grouped method, the grouping is made by Item Accounting Code that can be filled from each Item Page. It means that you can set your External Accounting ERP Codes in StoresAce Items and retrieve values grouped by each Code.

In case Items Accounting codes are not filled, the Grouped Methods return a default group having the Item' Classification and Vat Details.

Purchases [Full]	Purchases Movements with full Details Information
Purchases [Grouped]	Purchases Movements Details are grouped with totals for each Document. The grouping is made by Item Accounting Code that can be filled from each Item Page. It means that you can set your External Accounting ERP Codes in StoresAce Items and retrieve values grouped by each Code.

Important Considerations:

- For the Purchases Movements, the StoresAce Accounting Module is required.
- Only movements and values set as Conferred are displayed in these methods.
- Only Closed documents are listed in the API (Cancelled, Draft and Open documents are excluded).
- Everytime a Purchase Document is modified, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Purchases Methods to filter results:

Argument	Type	Details
TaxRegistrationNumber	Text	Company Tax Number to filter Purchases from all Company' Stores
store	Text	Store Code to filter Purchases from a particular Store
accounting_store	Text	Store Accounting Code to filter Sales from a particular Store
date	Date (YYYY-MM-DD)	Filter Purchases Movements in a particular Day
SupplierTaxId	Text	Filter by Supplier Tax Number
InvoiceNo	Text	Filter by Invoice Purchase Number. Must pass the exact full document number.

The Output in Json format for the Full Details is as follows:

```
[{
  "Id": 12745,
  "TransId": 1,
  "InvoiceNo": "706943",
  "AccountingDate": "2016-07-19",
  "InvoiceDate": "2016-07-06",
  "InvoiceTime": "2016-07-19T16:29:40",
  "SystemEntrydate": "2016-07-19T16:29:40",
  "SystemEntryDateUser": null,
  "SupplierId": "200108",
  "SupplierCompanyName": "Fornecedor 200108",
  "SupplierTaxId": "929732462",
  "SupplierAddressDetail": "Rua 17",
  "SupplierPostalCode": "4017",
  "SupplierCity": "Setúbal",
  "SupplierCountry": "PRT",
  "ShipFromAddressDetail": "Rua 17",
  "ShipFromCity": "Setúbal",
  "ShipFromPostalCode": "4017",
  "ShipFromCountry": "PRT",
  "ShipToDeliveryDate": "2016-07-19",
  "ShipToAddressDetail": "Rua 70",
  "ShipToAddressPostalCode": "2070",
  "ShipToAddressCity": "Leiria",

  /*StoresAce Unique Integer Identifier*/
  /*Store Movement Purchase Code */
  /*Purchase Document Number (Supplier Invoice No)*/
  /*Conferred Date*/
  /*Supplier Invoice Ref. Document Date*/
  /*Created Date into StoresAce*/
  /*Created Date into StoresAce*/
  /*Created By StoresAce User*/
  /*StoresAce Supplier Code*/
  /*Supplier Name*/
  /*Supplier Tax Number*/
  /*Supplier Address*/
  /*Supplier Postal Code*/
  /*Supplier City*/
  /*Supplier Country*/
  /*From Address*/
  /*From City*/
  /*From Postal Code*/
  /*From Country*/
  /*Delivery Date*/
  /*To Address*/
  /*To Postal Code*/
  /*To City*/
}
```

```

"ShipToAddressCountry": "PRT",
"ActualDeliveryDate": "2016-07-19",
"TaxRegistrationNumber": "599999993",
"TotalTransactionAmount": 112.29,
"TotalTaxAmount": 21.0,
"TotalNetAmount": 91.29,
"TotalLineItemDiscountAmount": 0.0,
"PaymentTermsCode": 2,
"PaymentTerms": "A 30 Dias",
"PaymentTermsDays": 30,
"PaymentDeadline": "2016-08-05",
"PaymentDate": "2016-07-19",
"TotalPaidAmount": 112.29,
"TotalPaymentDiscountAmount": 0.0,
"synccounter": 0,
"CompanyName": "DEMO",
"CompanyTaxNumber": "599999993",
"StoreId": "70",
"StoreName": "Leiria",
"StoreExternalCode": null,
"StoreSector": null,
"StoreAccountingCode": null,
"TransDocument": "FCO",
"TransDocumentDescription": "Fatura",
"TransNature": null,
"NatureDescription": null,
"TransactionTaxIncluded": false,
"BuyTransactionDetails": [
    {
        "ItemId": "16964",
        "ProductDescription": "Artigo 16964",
        "Warehousel": "17",
        "Quantity": 1.0,
        "UnitPrice": 23.4,
        "UnitOfSaleId": "Caixa 20 unid",
        "TaxCode": "NOR",
        "TaxPercentage": 23.0,
        "TaxCountryRegion": "PT",
        "TotalNetAmount": 23.4,
        "TotalTaxAmount": 5.382,
        "TotalOtherTaxAmount": 0.0,
        "TotalLineItemDiscountAmount": 0.0,
        "TotalTaxIncludedAmount": 28.782,
        "DiscountPercent": 0.0
    }
]
}
}

```

```

/*To Country*/
/*Delivery Date*/
/*Store Company Tax Number*/
/*Total Final Document value with Vat*/
/*Total Vat amount*/
/*Total Liquid amount without Vat*/
/*Discount Total*/
/*StoresAce Payment Terms Code*/
/*Payment Terms Description*/
/*Payment Terms Days*/
/*Payment Deadline*/
/*Paid Date*/
/*Paid Amount*/
/*Payment Discount*/
/*Document EPOCH Synccounter*/
/*Store Company Name*/
/*Company Tax Number*/
/*StoresAce Store Code*/
/*Store Name*/
/*Store External Code*/
/*Store Accounting Sector Code*/
/*Store Accounting Code*/
/*Document Type*/
/*Document Description*/
/*Document Nature*/
/*Nature Description*/
/*True if details Unit Prices are with Vat Included*/
/*Array with all Purchase Invoice Details*/

/*StoresAce Item Code*/
/*StoresAce Item Name*/
/*StoresAce Entry/Exit Warehouse Code*/
/*Quantity*/
/*Unit Price*/
/*StoresAce Unit Abbreviation*/
/*Vat Tax Type*/
/*Vat Tax Percent*/
/*Vat Tax Region*/
/*Total Liquid Amount without Vat */
/*Total Vat Amount*/
/*Total Other Taxes Amount (IEC, Green Point, etc)*/
/*Total Line Discount Amount*/
/*Total Final Line Amount with Vat*/
/*Discount Percent*/

```

Deliveries Movements

With this Web Service you can access the full list of Deliveries and Returns Movements from stores, including all details.

It contains full Deliveries Movements Details, like nature, total price, tax values, supplier identification, date and time, products quantities and purchased prices details. This will consider only delivery or transport movements made to the stores, but not the final invoices. For final invoices, please consider the Purchases Movements API.

```
"Deliveries [Full]": "https://demo.storesace.com/dimas/api/headoffice/deliveries/full/?format=json&page_size=10",
```

Important Considerations:

- Only Closed documents are listed in the API (Cancelled, Draft and Open documents are excluded).
- Everytime a Purchase Document is modified, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Deliveries Methods to filter results:

Argument	Type	Details
TaxRegistrationNumber	Text	Company Tax Number to filter Deliveries from all Company' Stores
store	Text	Store Code to filter Deliveries from a particular Store
accounting_store	Text	Store Accounting Code to filter Sales from a particular Store
date	Date (YYYY-MM-DD)	Filter Deliveries Movements in a particular Day
SupplierTaxId	Text	Filter by Supplier Tax Number
InvoiceNo	Text	Filter by Document Number. Must pass the exact full document number.

The Output in Json format for the Full Details is as follows:

```
[{
  "Id": 1001,
  "synccounter": 1670417515,
  "TransId": 1,
  "InvoiceNo": "706943",
  "InvoiceDate": "2016-07-06",
  "InvoiceTime": "2016-07-19T16:29:40",
  "SystemEntrydate": "2016-07-19T16:29:40",
  "SystemEntryDateUser": null,
  "SupplierId": "200108",
  "SupplierCompanyName": "Fornecedor 200108",
  "SupplierTaxId": "929732462",
  "SupplierAddressDetail": "Rua 17",
  "SupplierPostalCode": "4017",
  "SupplierCity": "Setúbal",
  "SupplierCountry": "PRT",
  "ShipFromAddressDetail": "Rua 17",
  "ShipFromCity": "Setúbal",
  "ShipFromPostalCode": "4017",
  "ShipFromCountry": "PRT",
  "ShipToDeliveryDate": "2016-07-19",
  "ShipToAddressDetail": "Rua 70",
  "ShipToAddressPostalCode": "2070",
  "ShipToAddressCity": "Leiria",
  "ShipToAddressCountry": "PRT",
  "ActualDeliveryDate": "2016-07-19",
  "TaxRegistrationNumber": "599999993",
  "TotalTransactionAmount": 112.29,
  "TotalTaxAmount": 21.0,
  "TotalNetAmount": 91.29,
  "TotalLineItemDiscountAmount": 0.0,
  "CompanyName": "DEMO",
  "CompanyTaxNumber": "599999993",
  "StoreId": "70",
  "StoreName": "Leiria",
  "StoreExternalCode": null,
  "StoreSector": null,
  "StoreAccountingCode": null,
  "TransDocument": "GT",
  "TransDocumentDescription": "Guia Transporte",
  "TransNature": "GT",
  "NatureDescription": "Guia Transporte",
  "TransactionTaxIncluded": false,
  "BuyTransactionDetails": [
    {
      "ItemId": "16964",
      /*StoresAce Unique Integer Identifier*/
      /*Document EPOCH Synccounter*/
      /*Store Movement Delivery Code */
      /*Purchase Document Number (Supplier Invoice No)*/
      /*Supplier Invoice Ref. Document Date*/
      /*Created Date into StoresAce*/
      /*Created Date into StoresAce*/
      /*Created By StoresAce User*/
      /*StoresAce Supplier Code*/
      /*Supplier Name*/
      /*Supplier Tax Number*/
      /*Supplier Address*/
      /*Supplier Postal Code*/
      /*Supplier City*/
      /*Supplier Country*/
      /*From Address*/
      /*From City*/
      /*From Postal Code*/
      /*From Country*/
      /*Delivery Date*/
      /*To Address*/
      /*To Postal Code*/
      /*To City*/
      /*To Country*/
      /*Delivery Date*/
      /*Store Company Tax Number*/
      /*Total Final Document value with Vat*/
      /*Total Vat amount*/
      /*Total Liquid amount without Vat*/
      /*Discount Total*/
      /*Store Company Name*/
      /*Company Tax Number*/
      /*StoresAce Store Code*/
      /*Store Name*/
      /*Store External Code*/
      /*Store Accounting Sector Code*/
      /*Store Accounting Code*/
      /*Document Type*/
      /*Document Description*/
      /*Document Nature*/
      /*Nature Description*/
      /*True if details Unit Prices are with Vat Included*/
      /*Array with all Delivery Document Details*/
    }
  ]
}
```

```

    "ProductDescription": "Artigo 16964", /*StoresAce Item Name*/
    "WarehouseId": "17", /*StoresAce Entry/Exit Warehouse Code*/
    "Quantity": 1.0, /*Quantity*/
    "UnitPrice": 23.4, /*Unit Price*/
    "UnitOfSaleId": "Caixa 20 unid", /*StoresAce Unit Abbreviation*/
    "TaxCode": "NOR", /*Vat Tax Type*/
    "TaxPercentage": 23.0, /*Vat Tax Percent*/
    "TaxCountryRegion": "PT", /*Vat Tax Region*/
    "TotalNetAmount": 23.4, /*Total Liquid Amount without Vat */
    "TotalTaxAmount": 5.382, /*Total Vat Amount*/
    "TotalOtherTaxAmount": 0.0, /*Total Other Taxes Amount (IEC, Green Point, etc)*/
    "TotalLineItemDiscountAmount": 0.0, /*Total Line Discount Amount*/
    "TotalTaxIncludedAmount": 28.782, /*Total Final Line Amount with Vat*/
    "DiscountPercent": 0.0 /*Discount Percent*/
  }
}
}}
```

Stock Movements

With this Web Service you can access the full list of Stock Movements, including all details, or a resumed List where all details are grouped into totals for each Document Movement.

It contains full Movement Details with total price, tax values, stock entry and exit locations, date and time, products quantities and stock average/last cost prices details.

There are 10 API links for Purchases Information. Choose the correct one depending on which level of detail/grouped information you pretend to get:

```
"Warehouse Transfers [Full]": "https://demo.storesace.com/dimas/api/headoffice/warehousetransfers/full/?format=json&page_size=10",
"Warehouse Transfers [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/warehousetransfers/?format=json&page_size=10",
"Waste Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/wastemovements/full/?format=json&page_size=10",
"Waste Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/wastemovements/?format=json&page_size=10",
"Entry Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/entrymovements/full/?format=json&page_size=10",
"Entry Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/entrymovements/?format=json&page_size=10",
"Exit Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/exitmovements/full/?format=json&page_size=10",
"Exit Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/exitmovements/?format=json&page_size=10",
"Production Movements [Full]": "https://demo.storesace.com/dimas/api/headoffice/productionmovements/full/?format=json&page_size=10",
"Production Movements [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/productionmovements/?format=json&page_size=10",
```

Note: For any Grouped method, the grouping is made by Item Accounting Code that can be filled from each Item Page. It means that you can set your External Accounting ERP Codes in StoresAce Items and retrieve values grouped by each Code.

In case Items Accounting codes are not filled, the Grouped Methods return a default group having the Item' Classification and Vat Details.

Warehouse Transfers [Full]	Warehouse Transfers Stock Movements with full Details Information
Warehouse Transfers [Grouped]	Warehouse Transfers Stock Movements Details are grouped with totals for each Document.
Waste Movements [Full]	Warehouse Waste Movements with full Details Information
Waste Movements [Grouped]	Warehouse Waste Movements Details are grouped with totals for each Document.
Entry Movements [Full]	Warehouse Justified Entry Movements with full Details Information
Entry Movements [Grouped]	Warehouse Justified Entry Movements Details are grouped with totals for each Document.
Exit Movements [Full]	Warehouse Justified Exit Movements with full Details Information

Exit Movements [Grouped]	Warehouse Justified Exit Movements Details are grouped with totals for each Document.
Production Movements [Full]	Warehouse Production Movements with full Details Information
Production Movements [Grouped]	Warehouse Production Movements Details are grouped with totals for each Document.

Important Considerations:

- Only Closed documents are listed in the API (Cancelled, Draft and Open documents are excluded).
- Everytime a Stock Movement Document is modified, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Stock Movements Methods to filter results:

Argument	Type	Details
TaxRegistrationNumber	Text	Company Tax Number to filter Stock Movements from all Company' Stores
store	Text	Store Code to filter Stock Movements from a particular Store
accounting_store	Text	Store Accounting Code to filter Sales from a particular Store
date	Date (YYYY-MM-DD)	Filter Stock Movements in a particular Day

The Output in Json format for the Full Details is as follows:

```
[{
  "Id": 12745,
  "TransId": 2,
  "InvoiceNo": "GT H61/2",
  "InvoiceDate": "2016-07-14",
  "InvoiceTime": "2016-07-14T17:56:11",
  "MovementStartTime": "2016-07-14T18:31:48",
  "MovementEndTime": null,
  "CompanyName": "DEMO",
  "CompanyTaxNumber": "599999993",
  "StoreId": "61",
  "StoreName": "Braga",
  /*StoresAce Unique Integer Identifier*/
  /*Store Movement Document Code */
  /*Stock Movement Document Number*/
  /*Movement Document Date*/
  /*Created Date into StoresAce*/
  /*Transport Start Date and Time*/
  /*Transport End Date and Time*/
  /*Store Company Name*/
  /*Company Tax Number*/
  /*StoresAce Origin Store Code*/
  /*Origin Store Name*/
}
```



```

"StoreExternalCode": null,
"StoreSector": null,
"StoreAccountingCode": null,
"WarehouseId": "8",
"WarehouseName": "Armazém Braga (8)",
"ShipToStoreId": "63",
"ShipToStoreName": "Lisboa",
"ShipToWarehouseId": "10",
"ShipToWarehouseName": "Armazém Lisboa (10)",
"SoftwareCertificateNumber": "2499",
"Signature": "IV/EEEUEiqlOeS/aLSY9MY",
"KeyVersion": "1",
"TransSerial": "H61",
"TransDocNumber": 2,
"ShipFromAddressDetail": "Rua 61",
"ShipFromCity": "Braga",
"ShipFromPostalCode": "2061",
"ShipFromCountry": "PRT",
"ShipToAddressDetail": "Rua 63",
"ShipToAddressPostalCode": "2063",
"ShipToAddressCity": "Lisboa",
"ShipToAddressCountry": "PRT",
"ActualDeliveryDate": null,
"TaxRegistrationNumber": "500000000",
"TotalTransactionAmount": 2.02,
"TotalTaxAmount": 0.38,
"TotalNetAmount": 1.64,
"synccounter": 0,
"Createstamp": "2016-07-14T17:56:11",
"Updatestamp": "2016-10-13T12:34:20",
"TransactionTaxIncluded": false,
"TransDocument": "GT",
"TransDocumentDescription": "Guia de Transporte",
"TransNature": "GT",
"NatureDescription": "Guia de Transporte",
"TransactionDetails": [
    {
        "ItemId": "511237",
        "ProductDescription": "Artigo 511237",
        "LineNumber": 1,
        "Quantity": 7.0,
        "UnitPrice": 0.23437,
        "UnitId": "Unidade",
        "TaxCode": "NOR",
        "TaxPercentage": 23.0,
        "TaxCountryRegion": "PT",
        "TotalNetAmount": 1.64059,
        "TotalTaxIncludedAmount": 2.01793,
        /*Origin Store External Code*/
        /*Origin Store Accounting Sector Code*/
        /*Origin Store Accounting Code*/
        /*StoresAce Origin Exit Warehouse Code*/
        /*Origin Exit Warehouse Name*/
        /*Destination Store Code*/
        /*Destination Store Name*/
        /*Destination Entry Warehouse Code*/
        /*Destination Entry Warehouse Name*/
        /*Official Legal Certificate Number*/
        /*Document Signature*/
        /*Signature Key Version*/
        /*Transaction Serial*/
        /*Transaction Sequence Number*/
        /*Origin Address*/
        /*Origin City*/
        /*Origin Postal Code*/
        /*Origin Country*/
        /*Destination Address*/
        /*Destination Postal Code*/
        /*Destination City*/
        /*Destination Country*/
        /*Delivery Date and Time*/
        /*Store Company Tax Number*/
        /*Total Final Document value with Vat*/
        /*Total Vat amount*/
        /*Total Liquid amount without Vat*/
        /*Document EPOCH Synccounter*/
        /*Created Date and Time*/
        /*Last Updated Date and Time*/
        /*True if details Unit Prices are with Vat Included*/
        /*Transaction Document Type*/
        /*Transaction Document Description*/
        /*Transaction Nature*/
        /*Transaction Nature Description*/
        /*Array with all Transaction Details*/

        /*StoresAce Item Code*/
        /*StoresAce Item Name*/
        /*Document Line Number*/
        /*Quantity*/
        /*Unit Price*/
        /*StoresAce Unit Abbreviation*/
        /*Vat Tax Type*/
        /*Vat Tax Percent*/
        /*Vat Tax Region*/
        /*Total Liquid Amount without Vat */
        /*Total Final Line Amount with Vat*/
    }

```

```
}}}} "DiscountPercent": 0                /*Discount Percent*/
```

Inventories

With this Web Service you can access the full list of Inventory Stock Counts with a resumed list, where all details are grouped into totals for each Inventory Document.

It contains full Inventory information with total price, breaks in quantities and value, locations, date and time, products quantities and stock average/last cost prices details and totals.

```
"Inventories [Grouped]": "https://demo.storesace.com/dimas/api/headoffice/inventory/?format=json&page_size=10",
```

Note: For the available Grouped method, the grouping is made by Item Accounting Code that can be filled from each Item Page. It means that you can set your External Accounting ERP Codes in StoresAce Items and retrieve values grouped by each Code.

In case Items Accounting codes are not filled, the Grouped Methods return a default group having the Item' Classification.

Inventories [Grouped]	Warehouse Inventory Stock Count Details are grouped with totals for each Document.
-----------------------	--

Important Considerations:

- For the Inventory Movements, the StoresAce Accounting Module is required.
- Only inventories with values and quantities set as Conferred are displayed in these methods.
- Only Closed documents are listed in the API (Cancelled, Draft and Open documents are excluded).
- Everytime a Inventory Document is modified, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Inventory Method to filter results:

Argument	Type	Details
TaxRegistrationNumber	Text	Company Tax Number to filter Inventories from all Company' Stores
store	Text	Store Code to filter Inventories from a particular Store
accounting_store	Text	Store Accounting Code to filter Sales from a particular Store

date	Date (YYYY-MM-DD)	Filter Inventories in a particular Day
------	----------------------	--

The Output in Json format for the Grouped Details is as follows:

```
[{
  "Id": 12745,
  "TransId": 1,
  "TransDocNumber": 1,
  "Date": "2016-06-30",
  "CompanyName": "DEMO",
  "CompanyTaxNumber": "599999993",
  "StoreId": "61",
  "StoreName": "Braga",
  "StoreExternalCode": null,
  "StoreSector": null,
  "StoreAccountingCode": null,
  "WarehouseId": "8",
  "WarehouseName": "Armazém Braga (8)",
  "DiffQuantity": 505.5155,
  "DiffTotalAmount": 16.83,
  "TotalTransactionAmount": 13938.93,
  "TotalNetAmount": 13938.93,
  "synccounter": 0,
  "Createstamp": "2016-07-14T17:56:11",
  "Updatestamp": "2016-10-13T12:34:20",
  "TransactionTaxIncluded": false,
  "TransDocument": "INV",
  "TransDocumentDescription": "Inventory",
  "TransactionDetails": [
    {
      "ItemId": "M",
      "Quantity": 1,
      "UnitPrice": 13938.93213,
      "UnitOfSaleId": "UNI",
      "TotalNetAmount": 13938.93,
      "TotalTaxIncludedAmount": 13938.93
    }
  ]
}]
```

/*StoresAce Unique Integer Identifier*/
/*Store Inventory Document Code */
/*Store Inventory Document Code */
/*Inventory Date*/
/*Store Company Name*/
/*Company Tax Number*/
/*StoresAce Store Code*/
/*Store Name*/
/*Store External Code*/
/*Store Accounting Sector Code*/
/*Store Accounting Code*/
/*StoresAce Inventory Warehouse Code*/
/*Warehouse Name*/
/*Total Break Quantity (Real vs Theoric Quantity)*/
/*Total Break Value (Real vs Theoric Value)*/
/*Total Inventory Value (Sum of ACP or LCP)*/
/*Total Inventory Value (Sum of ACP or LCP)*/
/*Document EPOCH Synccounter*/
/*Created Date and Time*/
/*Last Updated Date and Time*/
/*True if details Unit Prices are with Vat Included*/
/*Transaction Document Type*/
/*Transaction Document Description*/
/*Array with Grouped Transaction Details*/
/*StoresAce Grouped Accounting Code*/
/*Total Quantity*/
/*Average Unit Price*/
/*Default Unit*/
/*Total Line Amount*/
/*Total Line Amount*/

Stores

With this Web Service you can access the full registered Stores information.

```
"Stores": "https://demo.storesace.com/dimas/api/headoffice/stores/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Store is created or an existing one is modified, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Stores Web Service to filter results:

Argument	Type	Details
TaxRegistrationNumber CompanyTaxNumber	Text	Company Tax Number to filter Inventories from all Company' Stores
CompanyName	Text	Company Name to filter Inventories from all Company' Stores
BrandId	Text	Filter Stores by Brand Code
BrandName	Text	Filter Stores by Brand Name
store StoreId	Text	Store Code to filter any particular Store
AccountingCode	Text	Store Accounting Code to filter any particular Store
ExternalCode	Text	Store External Code to filter any particular Store
Sector	Text	Store Sector to filter Stores by Sector
TaxRegion	Text	Filter Stores by Tax Region Identifier
TaxRegionId	Text	Filter Stores by Tax Region Code
TaxRegionName	Text	Filter Stores by Tax Region Name

Warehouses

With this Web Service you can access the full registered Warehouses information.

```
"Warehouses": "https://demo.storesace.com/dimas/api/headoffice/warehouses/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Warehouse is created or an existing one is modified, it gains a new Synccounter value with current EPOCH date and time.

Families

With this Web Service you can access the full registered Families and Subfamilies information.

```
"Families": "https://demo.storesace.com/dimas/api/headoffice/families/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Family is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```
{
  "Id": 12745,
  "FamilyId": 12001,
  "FamilyLevel": 2,
  "FamilyLevelId": 72,
  "Status": 1,
  "Inactive": false,
  "Discontinued": false,
  "FamilyFullPath": "CAFETARIA\\Cafés",
  "ParentFamilyId": 12000,
  "ParentFamilyLevelId": 11,
  "ParentFamilyDescription": "CAFETARIA",
  "FamilyDescription": "Cafés",
  /*StoresAce Unique Integer Identifier*/
  /*StoresAce Family Code*/
  /*Family Level (>1 are Subfamilies)*/
  /*Internal Unique Family Level Code - Integer Value*/
  /*Status: 1 - Active, 0 - Inactive*/
  /*Inactive Status: Inactive if true, Active if false*/
  /*Discontinued Status*/
  /*Full Family Named Path with all Parent Families*/
  /*Parent Family Code*/
  /*Parent Internal Unique Level Code*/
  /*Parent Family Name*/
  /*Family Name*/
}
```

```
"FamilyShortDescription": "Cafés",
"synccounter": 1615379496,
"Updatestamp": "2016-03-10T12:31:36",
"Createstamp": null,
"deleted": 0,
"Code": 12001,
"ShowInScreen": 1,
"PrintingOrder": null,
"DisplayOrder": 0,
"ButtonColor": "#000000",
"FontColor": null,
"Zones": []
}}
```

```
/*Family Short Name. Also Family Button Name on POS*/
/*Family EPOCH Synccounter*/
/*Last Updated Date and Time*/
/*Created Date and Time*/
/*Deleted Status: 1 - Deleted, 0 - Not Deleted*/
/*Family Code*/
/*Display on POS Screen*/
/*POS Printing Order*/
/*POS Display Order*/
/*POS Button HEX Color*/
/*POS Button Font HEX Color*/
/*Array with all POS Sale Zones associated with Family*/
```

Items

With this Web Service you can access the full registered Products information. The HeadOffice Items API WebService will return all Products available in StoresAce. Although, the POS API will return only the Products that need to be sent to POS Systems, so Generic, Purchase and Production item types are excluded.

```
"Items": "http://burgerking.localhost:8000/dimas/api/headoffice/items/?format=json&page_size=10",
"Items & Stores": "http://burgerking.localhost:8000/dimas/api/headoffice/itemsstores/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Item is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Items Web Service to filter results:

Argument	Type	Details
ItemId	Text	Filter by Item Code
BrandId	Text	Filter Items by Store's Brand Code
store	Text	Store Code to filter Items by Store
accounting_store	Text	Store Accounting Code to filter Items by Store

The Output in Json format is as follows:

```
{
  "Id": 12745,
  "ItemId": "031",
  "Status": 1,
  "Type": 1,
  "TypeDescription": "Sale",
  "ItemDescription": "Tosta Mista",
  "ItemPublicDescription": "Tosta Mista",
  "ItemShortDescription": "Tosta Mista",
  "ItemShortDescription2": "Tosta Mista",
  "ItemLongDescription": "",
  "ExternalCode": null,
  "ParentFamilyId": 4000,
  /*StoresAce Unique Integer Identifier*/
  /*Item Code (String)*/
  /*Item Status: 1 - Active, 0 - Inactive*/
  /*StoresAce Item Type*/
  /*StoresAce Item Type Description*/
  /*Item Name*/
  /*Item Public Name (For Customers Reading)*/
  /*Item Short Name and POS Button Name*/
  /*Item Short Name 2 and POS Button Name*/
  /*Item Long Description*/
  /*Item External Code*/
  /*BackOffice Parent Family Code*/
}
```


"ParentFamilyLevelId": 4,	/*BackOffice Parent Family Level Code*/
"ParentFamilyDescription": "SNACKS",	/*BackOffice Parent Family Name*/
"FamilyId": 4008,	/*BackOffice SubFamily Code*/
"FamilyLevel": 2,	/*BackOffice SubFamily Level*/
"FamilyLevelId": 48,	/*BackOffice SubFamily Level Code*/
"FamilyDescription": "Sandes",	/*BackOffice SubFamily Name*/
"FamilyFullPath": "SNACKS\\Sandes",	/*BackOffice Family Full Names Path*/
"POSParentFamilyId": 4000,	/*POS Parent Family Code*/
"POSParentFamilyLevelId": 4,	/*POS Parent Family Level Code*/
"POSParentFamilyDescription": "SNACKS",	/*POS Parent Family Name*/
"POSFamilyId": 4008,	/*POS SubFamily Code*/
"POSFamilyLevel": 2,	/*POS SubFamily Level*/
"POSFamilyLevelId": 48,	/*POS SubFamily Level Code*/
"POSFamilyDescription": "Sandes",	/*POS SubFamily Name*/
"POSFamilyFullPath": "SNACKS\\Sandes",	/*POS Family Full Names Path*/
"ParentGroupId": null,	/*Parent Category Code*/
"ParentGroupLevelId": null,	/*Parent Category Level Code*/
"ParentGroupDescription": null,	/*Parent Category Name*/
"GroupId": 19,	/*Category Code*/
"GroupLevel": 1,	/*Category Level*/
"GroupLevelId": 13,	/*Category Level Code*/
"GroupDescription": "Tostas",	/*Category Name*/
"GroupFullPath": "Tostas",	/*Category Full Names Path*/
"AccountingCode": null,	/*Item Accounting Code*/
"AccountingGroupId": null,	/*Accounting Group Code*/
"AccountingGroupDescription": null,	/*Accounting Group Name*/
"AccountingGroupFullPath": null,	/*Accounting Group Full Names Path*/
"DefaultSellingUnit": 1,	/*Default/Sales Unit ID*/
"DefaultSellingUnitId": "Unidade",	/*Default/Sales Unit Abbreviation*/
"DefaultBuyUnit": 1,	/*Purchase Unit ID*/
"DefaultBuyUnitId": "Unidade",	/*Purchase Unit Abbreviation*/
"DefaultStockUnit": 1,	/*Stock Unit Id*/
"DefaultStockUnitId": "Unidade",	/*Stock Unit Abbreviation*/
"UnitConversion": 1.0,	/*Unit Factor Conversion*/
"ProductCategory": "M",	/*Item SAFT Category*/
"ProductType": "P",	/*Item SAFT Type*/
"CreateDate": "2016-02-18T20:10:11",	/*Created Date and Time*/
"Barcode": "8808993834563",	/*Barcode*/
"ItemType": 1,	/*POS Item Type*/
"ReadQuantityFromScale": false,	/*POS - Read Quantity from Scale*/
"ExportToTouch": true,	/*POS - Display on Touch Screen*/
"TareValue": 0,	/*Tare Value*/
"PLU": 0,	/*POS - PLU Code*/
"MandatoryCustomer": false,	/*POS - Mandatory Customer Identification on Sales*/
"DoNotGroup": false,	/*POS - Group or Don't Group Quantities*/
"VariableSalePrice": false,	/*POS - True if Employee Can Change Sale Price*/
"CannotChangeSalePrice": true,	/*POS - True if Employee Can't Change Sale Price*/
"WeightUnitOfMeasure": "Unidade",	/*POS - Weight Unit Abbreviation*/

"WeightMeasure": null,	/*POS - Weight Measure Reference*/
"WeightMeasurePrice": 0,	/*POS - Sale Price at Weight Unit (Requires Store Argument)*/
"TagAlongItem": null,	/*POS - Associated Item Code*/
"TagAlongQuantity": null,	/*POS - Associated Item Quantity*/
"InformationMessage": null,	/*POS - Sale Information Display Message*/
"ButtonColor": "#337ab7",	/*POS - Button HEX Color Code*/
"FontColor": null,	/*POS - Button Font HEX Color Code*/
"MaxCookTime": 0,	/*POS - Maximum Cooking Time (minutes)*/
"AvgCookTime": 0,	/*POS - Average Cooking Time (minutes)*/
"ExcludeCompensation": 0,	/*POS - Exclude from Compensation*/
"PrintingOrder": null,	/*POS - Printing Order*/
"DisplayOrder": 50,	/*POS - Screen Display Order*/
"OptionsGroupId": null,	/*POS - Options Group Code*/
"MaxOptions": 1,	/*POS - Maximum of Options to Choose*/
"PrintLocationId": null,	/*POS - StoresAce Main Print Location Code*/
"PrintLocations": [],	/*POS - Array with Print Locations to print*/
"CommissionGroup": null,	/*POS - Commission Group Code*/
"BarcodeType": 0,	/*Barcode Type*/
"TaxType": 2,	/*Tax Type internal ID*/
"TaxableGroupId": 2,	/*Tax Type internal ID*/
"TaxTypeDescription": "Reduzida",	/*Tax Type Description*/
"TaxCode": "RED",	/*Tax Type Tax Code*/
"Tax": null,	/*Tax internal ID (Requires Store Argument)*/
"TaxRate": null,	/*Tax Percent (Requires Store Argument)*/
"Locked": 0,	/*POS - Locked Sale Item*/
"Discontinued": false,	/*POS - true if is Discontinued */
"StockManagement": false,	/*true if Item has Stock Management */
"Comments": "",	/*Item Comments Text*/
"LabelIngredients": "Fiambre fatia (20 g)",	/*Ingredients Information Text*/
"LabelNutrition": "",	/*Nutrition Information Text*/
"GrossWeight": 0.3,	/*Gross Weight*/
"LiquidWeight": 0.3,	/*Liquid Weight*/
"OriginCountryId": "",	/*Origin Country Code*/
"Image": "/9j/4AAQSkZJRgABAQ",	/*Base64 encoded main image*/
"BarcodeList": [],	/*List of all barcodes and units defined for this item*/
"SupplierId": "600061",	/*Main Supplier Code*/
"ItemSupplierList": [],	/*Suppliers Cardex List Details*/
"ItemComponents": [	/*Item Ingredients List (For Sale and Production Items)*/
{	
"ItemId": "031",	/*Current Item Code*/
"ComponentItemId": "0311",	/*Component/Ingredient Code*/
"Quantity": 0.08	/*Composition Quantity*/
}],	
"ItemColorList": [],	/*Item Colors List*/
"ItemSizeList": [],	/*Item Sizes List*/
"synccounter": 1617983684,	/*Item EPOCH Synccounter*/
"Updatestamp": "2016-04-09T15:54:45",	/*Last Updated Date and Time*/
"Createstamp": "2016-02-18T20:10:11",	/*Created Date and Time*/

"deleted": 0
}]

/*Deleted Status: 1 - Deleted, 0 - Not Deleted*/

Temporary Items

With this Web Service you can access and register temporary Products information. All registered information is stored in a temporary product list, which is pending to be completed and approved to be transferred to the official products list.

```
"Temporary Items": "https://demo.storesace.com/dimas/api/headoffice/temp/items/?format=json&page_size=10",
```

The Output in Json format is as follows:

```
{
  "Id": 1,
  "ItemId": "031",
  "Status": 1,
  "Type": 1,
  "TypeDescription": "Sale",
  "ItemDescription": "Tosta Mista",
  "ExternalCode": null,
  "ParentFamilyId": 4000,
  "ParentFamilyLevelId": 4,
  "ParentFamilyDescription": "SNACKS",
  "FamilyId": 4008,
  "FamilyLevel": 2,
  "FamilyLevelId": 48,
  "FamilyDescription": "Sandes",
  "FamilyFullPath": "SNACKS\\Sandes",
  "GroupId": 19,
  "GroupLevel": 1,
  "GroupLevelId": 13,
  "GroupDescription": "Tostas",
  "GroupFullPath": "Tostas",
  "AccountingCode": null,
  "DefaultSellingUnit": 1,
  "DefaultSellingUnitId": "Unidade",
  "Barcode": "8808993834563",
  "TaxType": 2,
  "TaxableGroupId": 2,
  "TaxTypeDescription": "Reduzida",
  "TaxCode": "RED",
  "Tax": null,
  "TaxRate": null,
  "SupplierId": "600061",
  "UnitOfMeasureId": "Unidade",
  "CostPrice": 12.99,
  "OrderMinValue": 0,

  /*StoresAce Unique Integer Identifier*/
  /*Item Code (String)*/
  /*Item Status: 1 - Active, 0 - Inactive*/
  /*StoresAce Item Type*/
  /*StoresAce Item Type Description*/
  /*Item Name*/
  /*Item External Code*/
  /*BackOffice Parent Family Code*/
  /*BackOffice Parent Family Level Code*/
  /*BackOffice Parent Family Name*/
  /*BackOffice SubFamily Code*/
  /*BackOffice SubFamily Level*/
  /*BackOffice SubFamily Level Code*/
  /*BackOffice SubFamily Name*/
  /*BackOffice Family Full Names Path*/
  /*Category Code*/
  /*Category Level*/
  /*Category Level Code*/
  /*Category Name*/
  /*Category Full Names Path*/
  /*Item Accounting Code*/
  /*Default/Sales Unit ID*/
  /*Default/Sales Unit Abbreviation*/
  /*Barcode*/
  /*Tax Type internal ID*/
  /*Tax Type internal ID*/
  /*Tax Type Description*/
  /*Tax Type Tax Code*/
  /*Tax internal ID (Requires Store Argument)*/
  /*Tax Percent (Requires Store Argument)*/
  /*Supplier Code*/
  /*Supplier Purchase Unit*/
  /*Supplier Unit Cost Price*/
  /*Minimum Order Amount*/
}
```

```
"OrderMaxValue": 0,  
"OrderMultipleValue":  
"AvailableOrders": true,  
"GrossWeight": 0.3,  
"LiquidWeight": 0.3,  
"Updatestamp": "2016-04-09T15:54:45",  
"Createstamp": "2016-02-18T20:10:11",  
"deleted": 0  
}}
```

```
/*Maximum Order Amount*/  
/*Multiple Order Amount*/  
/*Available for Orders*/  
/*Gross Weight*/  
/*Liquid Weight*/  
/*Last Updated Date and Time*/  
/*Created Date and Time*/  
/*Deleted Status: 1 - Deleted, 0 - Not Deleted*/
```

Suppliers

With this Web Service you can access the full registered Suppliers information.

```
"Suppliers": "https://demo.storesace.com/dimas/api/headoffice/suppliers/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Supplier is created or an existing one is modified and then marked to Synchronise, it gains a new Synccounter value with current EPOCH date and time.

There are also additional GET arguments you can pass to Deliveries Methods to filter results:

Argument	Type	Details
TaxRegistrationNumber	Text	Filter Suppliers associated to the Company Tax Number
store	Text	Store Code to filter Suppliers from a particular Store
accounting_store	Text	Store Accounting Code to filter Suppliers by Store
FederalTaxId	Text	Filter by Supplier Tax Number
SupplierId	Text	Filter by Supplier Code

The Output in Json format is as follows:

```
{
  "Id": 12745,
  "SupplierId": "200035",
  "FederalTaxId": "979538204",
  "SupplierName": "Fornecedor 200035",
  "Fax": null,
  "AddressLine1": "Rua 10",
  "AddressLine2": null,
  "PostalCode": "4010-034",
  "EmailAddress": "orders@storesace.com",
  "Comments": null,
  "PostalCodeName": "Coimbra",
  "Telephone1": null,
  "MobileTelephone1": null,
  /*StoresAce Unique Integer Identifier*/
  /*Supplier Code*/
  /*Tax Number*/
  /*Name*/
  /*FAX Number*/
  /*Address Line 1*/
  /*Address Line 2*/
  /*Address Postal Code*/
  /*Email*/
  /*Comments Text*/
  /*Location Name*/
  /*Telephone*/
  /*Mobile Telephone*/
}
```

Lisboa
Av. Eng. Arantes e Oliveira, 3 R/C, Lisboa, Portugal

Porto / Head Office
Rua Fernando Namora 117, Porto, Portugal

info@storesace.com

For the ones with multi-stores
Anyplace, anytime and anywhere.

Level up!
storesace.com

```
"WebAddress": "www.storesace.com",  
"CountryId": "PRT",  
"TaxRegionId": "PT",  
"PaymentId": 3,  
"ContactName": "Ivo Marinho",  
"Inactive": false,  
"Status": 1,  
"deleted": 0,  
"synccounter": 1562689257,  
"Updatestamp": "2016-08-05T14:32:46",  
"Createstamp": "2016-07-09T16:20:58"  
}}
```

```
/*Website URL*/  
/*Country Code*/  
/*VAT Region Code*/  
/*Payment Terms Code*/  
/*Contact Name Info*/  
/*true if Inactive, false if Active*/  
/*Status: 1 if Active, 0 if Inactive*/  
/*Deleted Status: 1 if Deleted, 0 if not Deleted*/  
/*Item EPOCH Synccounter*/  
/*Last Updated Date and Time*/  
/*Created Date and Time*/
```

Customers

With this Web Service you can access the full registered Customers information.

```
"Customers": "https://demo.storesace.com/dimas/api/headoffice/customers/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Customer is created or an existing one is modified and then marked to Synchronise, it gains a new Synccounter value with current EPOCH date and time.
- Customers are valid for all Stores. There is no relationship between Customers and Stores, and so they are valid in every store. Sending argument store doesn't affect the returned results.

There are also additional GET arguments you can pass to Deliveries Methods to filter results:

Argument	Type	Details
FederalTaxId	Text	Filter by Supplier Tax Number
CustomerId	Number	Filter by Customer Code
CardId	Text	Filter by Loyalty Card Number
MobileTelephone1	Text	Filter by Mobile Phone
EmailAddress	Text	Filter by Email Address

The Output in Json format is as follows:

```
[{
  "Id": 1,                                /*StoresAce Unique Integer Identifier*/
  "CustomerId": 121038,                   /*Customer Code (Number)*/
  "FederalTaxId": "908434601",           /*Customer Tax Number*/
  "EmailAddress": "email@storesace.com", /*Email*/
  "PersonalIdNumber": null,              /*Citizen Number*/
  "CustomerGroupDescription": null,      /*Group Description*/
  "CardId": "994626979",                 /*Loyalty Card Number*/
  "Telephone1": null,                    /*Telephone*/
  "MobileTelephone1": "994626979",       /*Mobile Telephone*/
}
```



```
"Fax": null,
"BirthDate": "1988-03-18",
"PriceLineId": null,
"PaymentId": null,
"PaymentTerms": null,
"WebAddress": null,
"OrganizationName": "Afonso Cruz",
"SalesmanId": null,
"AddressLine1": null,
"AddressLine2": null,
"PostalCode": null,
"PostalCodeName": "Bragança",
"CountryId": "PRT",
"ISOCountryId": "PT",
"CountryName": "Portugal",
"Comments": null,
"TotalPoints": 0,
"Balance": null,
"GlobalDiscount": null,
"DirectDiscount": null,
"LimitPurchaseValue": 0,
"LimitPurchaseDays": 0,
"LimitType": 0,
"synccounter": 1562689247,
"Updatestamp": "2016-04-09T15:54:45",
"Createstamp": "2016-02-18T20:10:11",
"deleted": 0
}}
```

```
/*Fax*/
/*Birth Date*/
/*POS - Sale Price Type (1 to 5)*/
/*POS - Payment Terms Code*/
/*POS - Payment Terms Description*/
/*Customer Website*/
/*Customer Name*/
/*Associated Employee Code*/
/*Address Line 1*/
/*Address Line 2*/
/*Postal Code*/
/*Address Location Name*/
/*Country Code*/
/*Country ISO Code*/
/*Country Name*/
/*Comments Text*/
/*Points*/
/*Balance Amount*/
/*POS - Global Sales Discount*/
/*POS - Direct Discount*/
/*POS - Purchase Amount Limit*/
/*POS - Purchase Days Limit*/
/*POS - Limit Type*/
/*Customer EPOCH Synccounter*/
/*Last Updated Date and Time*/
/*Created Date and Time*/
/*Deleted Status: 1 - Deleted, 0 - Not Deleted*/
```

Employees (POS Salesman)

With this Web Service you can access the full registered Employees information.

```
"Salesmans": "https://demo.storesace.com/dimas/api/headoffice/salemsans/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Salesman is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```
[{
  "Id": 1004,                                /*StoresAce Unique Integer Identifier*/
  "SalesmanId": "18",                        /*Salesman Code*/
  "ExternalCode": "",                       /*Salesman External Code*/
  "SalesmanName": "Tiago Sousa",            /*Full Name*/
  "SalesmanShortName": "Tiago Sousa",       /*Short Name and POS Button Name*/
  "AddressLine1": "",                      /*Address Line 1*/
  "AddressLine2": "",                      /*Address Line 2*/
  "PostalCode": "",                        /*Postal Code*/
  "CountryId": null,                       /*Country Code*/
  "Locality": "",                          /*Location Name*/
  "Telephone1": "",                       /*Telephone Number*/
  "MobilePhone1": "",                     /*Mobile Telephone Number*/
  "EmailAddress": "email@storesace.com",    /*Email*/
  "WebAddress": "www.storesace.com",        /*Website*/
  "AdmissionDate": null,                   /*Date of Admission*/
  "DismissalDate": null,                   /*Date of Dismissal*/
  "Birthdate": null,                      /*Date of Birth*/
  "Languageld": 1,                         /*POS - Language Id*/
  "SpokenLanguageld": "PT",               /*POS - Language Code*/
  "IdentityCard": "9597857",               /*Citizen Card Number*/
  "FederalTaxId": "959797778",             /*Tax Number*/
  "SalesmanPasswordE": "2222",             /*POS - Access Password*/
  "Comments": "",                         /*Comments Text*/
  "SalesGroupld": null,                   /*POS - Group Code*/
  "ApplyRetentionTax": null,               /*POS - Applies Retention*/
  "RetentionTax": null,                   /*POS - Retention Tax*/
  "CardId": "7260",                       /*POS - Card Number*/
  "DefaultMainTillId": null,               /*POS - Main Till Code*/
  "Inactive": false,                      /*Is Inactive: True - Inactive, False - Active*/
  "SessionId": null,                      /*POS - Session Id*/
}
```

```

"SelfExpenditureMin": null,
"SelfExpenditureLimit": null,
"SelfExpenditurePriceLineId": 0,
"SelfExpenditurePercent": null,
"SelfExpenditureQtyLimit": null,
"ComissionGroupId": null,
"ComissionGroupDescription": null,
"ComissionOnSale": null,
"ComissionOnPay": null,
"ComissionAccountTypeId": null,
"ComissionLimit": null,
"ComissionLimitTransactions": null,
"MaximumDiscountRate": 100.0,
"MinimumDiscountValue": null,
"DiscountPriceLineId": 0,
"DiscountPricePercent": null,
"DefaultTable": 0,
"TableRange": "100",
"DefaultScreen": 3,
"DefaultWorkScreen": 4,
"DefaultExitScreen": 0,
"SessionCloseAccount": null,
"Status": 1,
"PermissionsGroupId": 7,
"PermissionsGroupDescription": "Operador",
"ButtonColor": "#ffffff",
"AppGroups": [
{
    "AppGroupId": 7,
    "AppGroupDescription": "Operador"
}
],
"synccounter": 1617983684,
"Updatestamp": "2016-04-09T15:54:45",
"Createstamp": "2016-02-18T20:10:11",
"deleted": 0
}
}

/*POS - Consumption Minimum Amount*/
/*POS - Consumption Limit Amount*/
/*POS - Consumption Sale Price (1 to 5)*/
/*POS - Consumption Percent*/
/*POS - Consumption Limit Quantity*/
/*POS - Commission Group Code*/
/*POS - Commission Group Description*/
/*POS - Commission on Sale*/
/*POS - Commission on Payment*/
/*POS - Commission Account*/
/*POS - Commission Limit Amount*/
/*POS - Commission Limit Sales Transactions*/
/*POS - Maximum Discount Percent*/
/*POS - Minimum Discount Value*/
/*POS - Discount Sale Price (1 to 5)*/
/*POS - Discount Percent*/
/*POS - Default Sale Table*/
/*POS - Table Access Range*/
/*POS - Default Sale Screen*/
/*POS - Default Work Screen*/
/*POS - Default Exit Screen*/
/*POS - Close Session Account*/
/*Salesman Status: 1 - Active, 0 - Inactive*/
/*StoresAce Access Level Code (Permissions Group)*/
/*Access Level Name*/
/*POS - Button HEX Color*/
/*List of Access Levels (Can be multiple)*/

/**StoresAce Access Level Code (Permissions Group)*/
/*Access Level Name*/

/*Salesman EPOCH Synccounter*/
/*Last Updated Date and Time*/
/*Created Date and Time*/
/*Deleted Status: 1 - Deleted, 0 - Not Deleted*/

```

Payment Methods

With this Web Service you can access the full registered Payment Methods Types information.

"Payment Types": "https://demo.storesace.com/dimas/api/headoffice/paymenttypes/?format=json&page_size=10",

Important Considerations:

- Everytime a new Payment Method is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```
[{
  "Id": 12745,                /*StoresAce Unique Integer Identifier*/
  "TenderId": 1,             /*Payment Method Code*/
  "TenderName": "Dinheiro",   /*Full Name*/
  "TenderShortName": "MO",    /*Short Name*/
  "MaximumAcceptAmount": 0.0, /*POS - Maximum Limit Amount */
  "MinimumSaleAmount": 0.0,   /*POS - Available on Sales above*/
  "CanReturnMoneyChange": false, /*POS - Exchange in Cash*/
  "TenderUse": 2,             /*Applies to [0 - Income; 1 - Payments; 2 - Both]*/
  "DocIdUsedToPay": null,     /*POS - Document Type used for Payment*/
  "UseDocToPay": false,       /*True if DocIdUsedToPay is not Null*/
  "TenderType": 1,           /*POS - Payment Type*/
  "CustomerIdentificationRequiredFlag": false, /*POS - Requires Customer Identification*/
  "AccountIdentificationRequiredFlag": false, /*POS - Forces you to fill in check data*/
  "OpenCashDrawerRequiredFlag": false, /*POS - Open the drawer*/
  "UpdateTillReport": false, /*POS - Considered for cash balance*/
  "DebitedOnAccount": false, /*POS - Generates pending account*/
  "StampDutyBillOfExchange": null, /*POS - Stamp Bill Type*/
  "SaftPaymentMechanism": "NU", /*POS - SAFT Code*/
  "Inactive": true,          /*true if Inactive, false if Active*/
  "Status": 0,               /*Payment Method Status: 1 - Active, 0 - Inactive*/
  "synccounter": 1617983684, /*Payment Method EPOCH Synccounter*/
  "Updatestamp": "2016-04-09T15:54:45", /*Last Updated Date and Time*/
  "Createstamp": "2016-02-18T20:10:11", /*Created Date and Time*/
  "deleted": 0               /*Deleted Status: 1 - Deleted, 0 - Not Deleted*/
}]
```

Units

With this Web Service you can access the full registered Units information and their relationships.

```
"Units": "https://demo.storesace.com/dimas/api/headoffice/units/?format=json&page_size=10",
```

Important Considerations:

- Everytime a new Unit is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```
[{
  "Id": 38,                                /*StoresAce Unique Integer Identifier*/
  "Status": 1,                             /*Status: 1 if Active, 0 if Inactive*/
  "UnitOfMeasureId": "Caixa 8 unid",        /*Unit Abbreviation*/
  "UnitOfMeasureDescription": "Caixa 8 unid", /*Unit Full Name*/
  "ConversionTo": "Unidade",               /*Unit Converting to Abbreviation*/
  "UnitOfMeasureConversion": 8.0,          /*Unit Converting Factor*/
  "MaximumDecimals": 0,                    /*Maximum Number of Decimals*/
  "deleted": 0,                            /*Deleted Status: 1 if Deleted, 0 if not Deleted*/
  "synccounter": 1562689257,                /*Item EPOCH Synccounter*/
  "Updatestamp": "2016-08-05T14:32:46",     /*Last Updated Date and Time*/
  "Createstamp": "2016-07-09T16:20:58"     /*Created Date and Time*/
}]
```

POS API

The POS Root contains all methods used to integrate with any Point of Sale Software or any other Sales External System, like Ecommerce or Online Shops. It allows retrieving mainly the products, families, employees and prices needed for generating Sales in every defined store in StoresAce.

As it is the aim of a Point of Sale, it doesn't need to have the entire list of defined Entities. For this, StoresAce has the capacity to output only each Store associated results for Sales Purposes. So to work with these methods, the store argument is mandatory to filter the specific results that apply to each store.

The Point of Sale API Root is available at:

[https://\[environment\].storesace.com/dimas/api/pos/](https://[environment].storesace.com/dimas/api/pos/)

```
{
  "Stores": "https://demo.storesace.com/dimas/api/pos/stores/?format=json&page_size=10",
  "Warehouses": "https://demo.storesace.com/dimas/api/pos/warehouses/?format=json&page_size=10",
  "Items": "https://demo.storesace.com/dimas/api/pos/items/?format=json&page_size=10",
  "Items & Prices": "https://demo.storesace.com/dimas/api/pos/itemsfull/?format=json&page_size=10",
  "Menu Items": "https://demo.storesace.com/dimas/api/pos/menuitems/?format=json&page_size=10",
  "ExtraProperties": "https://demo.storesace.com/dimas/api/pos/extraproperties/",
  "Stocks": "https://demo.storesace.com/dimas/api/pos/stocks/?format=json&page_size=10",
  "Item Line Prices": "https://demo.storesace.com/dimas/api/pos/itemprices/?format=json&page_size=10",
  "Sales Prices": "https://demo.storesace.com/dimas/api/pos/salesprices/?format=json&page_size=10",
  "Families": "https://demo.storesace.com/dimas/api/pos/families/?format=json&page_size=10",
  "Groups": "https://demo.storesace.com/dimas/api/pos/groups/?format=json&page_size=10",
  "Suppliers": "https://demo.storesace.com/dimas/api/pos/suppliers/?format=json&page_size=10",
  "SalesMans": "https://demo.storesace.com/dimas/api/pos/salesmans/?format=json&page_size=10",
  "SalesMan Levels": "https://demo.storesace.com/dimas/api/pos/salesmanlevels/?format=json&page_size=10",
  "Customers": "https://demo.storesace.com/dimas/api/pos/customers/?format=json&page_size=10",
  "CashDrawers (Till)": "https://demo.storesace.com/dimas/api/pos/cashdrawers/?format=json&page_size=10",
  "CashDrawers Movements (TenderTransaction)": "https://demo.storesace.com/dimas/api/pos/cashdrawersmovements/?format=json&page_size=10",
  "Payment Types (Tender)": "https://demo.storesace.com/dimas/api/pos/paymenttypes/?format=json&page_size=10",
  "Zones (Locals)": "https://demo.storesace.com/dimas/api/pos/zones/?format=json&page_size=10",
  "Terminals (Workstations)": "https://demo.storesace.com/dimas/api/pos/salesterminals/?format=json&page_size=10",
  "Print Locations": "https://demo.storesace.com/dimas/api/pos/printlocations/?format=json&page_size=10",
  "Exclusions (RAW)": "https://demo.storesace.com/dimas/api/pos/exclusionsraw/?format=json&page_size=10",
  "Payment Terms": "https://demo.storesace.com/dimas/api/pos/paymentsterms/?format=json&page_size=10",
  "Tax Types": "https://demo.storesace.com/dimas/api/pos/taxtypes/?format=json&page_size=10",
  "Tax Regions": "https://demo.storesace.com/dimas/api/pos/taxregions/?format=json&page_size=10",
  "Taxes": "https://demo.storesace.com/dimas/api/pos/taxes/?format=json&page_size=10",
  "Countries": "https://demo.storesace.com/dimas/api/pos/countries/?format=json&page_size=10",
  "Currencies": "https://demo.storesace.com/dimas/api/pos/currencies/?format=json&page_size=10",
  "Units": "https://demo.storesace.com/dimas/api/pos/units/?format=json&page_size=10",
  "Purchases [Full]": "https://demo.storesace.com/dimas/api/pos/purchases/full/?format=json&page_size=10",
  "Settlements [Full]": "https://demo.storesace.com/dimas/api/pos/settlements/?format=json&page_size=10",
  "Sales [Full]": "https://demo.storesace.com/dimas/api/pos/sales/?format=json&page_size=10",
  "Mix&Match - Campaigns List": "https://demo.storesace.com/dimas/api/pos/campaigns/?format=json&page_size=10",
  "Mix&Match - Price Discounts List": "https://demo.storesace.com/dimas/api/pos/pricediscounts/?format=json&page_size=10",
  "Loyalty - Check": "https://demo.storesace.com/dimas/api/pos/loyalty/sale/check/?format=json",
  "Loyalty - Rescue": "https://demo.storesace.com/dimas/api/pos/loyalty/sale/rescue/?format=json",
  "Loyalty - Customer Points": "https://demo.storesace.com/dimas/api/pos/customers/points/get/?format=json"
}
```

Sales Transactions

With this Web Service you can access the full list of Sales Transactions, including all details. Use this method to POST the Sales Transactions made on Point of Sales.

The Web Service URL used to both GET and POST results is available at:

[https://\[environment\].storesace.com/dimas/api/pos/sales/](https://[environment].storesace.com/dimas/api/pos/sales/)

Check the [Sales \[Full\]](#) definition described in HeadOffice methods for more information about this service.

Stores

With this Web Service you can access the full registered Stores information.

[https://\[environment\].storesace.com/dimas/api/pos/stores/](https://[environment].storesace.com/dimas/api/pos/stores/)

Check the [Stores](#) definition described in HeadOffice methods for more information about this service.

Warehouses

With this Web Service you can access the full registered Warehouses information.

[https://\[environment\].storesace.com/dimas/api/pos/warehouses/](https://[environment].storesace.com/dimas/api/pos/warehouses/)

Check the [Warehouses](#) definition described in HeadOffice methods for more information about this service.

Families

With this Web Service you can access the Stores' Families and Subfamilies information.

[https://\[environment\].storesace.com/dimas/api/pos/families/](https://[environment].storesace.com/dimas/api/pos/families/)

Check the [Families](#) definition described in HeadOffice methods for more information about this service.

Items

With this Web Service you can access the Stores' Sales Items information. This method will only return Items available for Sale at each Store. It means that only Sale, Purchase&Sale and Production&Sale Items Types will be returned by this method.

[https://\[environment\].storesace.com/dimas/api/pos/items/](https://[environment].storesace.com/dimas/api/pos/items/)

```
"Items": "http://burgerking.localhost:8000/dimas/api/pos/items/?format=json&page_size=10",
"Items & Prices": "http://burgerking.localhost:8000/dimas/api/pos/itemsfull/?format=json&page_size=10",
"Items & Stores": "http://burgerking.localhost:8000/dimas/api/pos/itemsstores/?format=json&page_size=10",
"Items Raw & Prices": "http://burgerking.localhost:8000/dimas/api/pos/itemsraw/?format=json&page_size=10",
```

Check the [Items](#) definition described in HeadOffice methods for more information about this service.

Items & Prices

With this Web Service you can access the Stores' Sales Items information with Sales Prices and Taxes.

This method is an extension of the [POS Items](#) method.

[https://\[environment\].storesace.com/dimas/api/pos/itemsfull/](https://[environment].storesace.com/dimas/api/pos/itemsfull/)

As an extension of the previous service the following additional information is available in this method, when passing the store argument:

"SalesPricesId": 1,	/*Sales Price Table Internal Identifier*/
"SalesPricesName": "Geral",	/*Sales Price Table Name*/
"TaxIncludedPrice1": 3.3,	/*Sale Price 1 - With VAT included*/
"TaxIncludedPrice2": 4.29,	/*Sale Price 2 - With VAT included*/
"TaxIncludedPrice3": 4.4,	/*Sale Price 3 - With VAT included*/
"TaxIncludedPrice4": 4.62,	/*Sale Price 4 - With VAT included*/
"TaxIncludedPrice5": 4.07,	/*Sale Price 5 - With VAT included*/
"TaxType1": 3,	/*Sale Tax Type 1 Internal Identifier*/
"TaxType1Description": "Intermédia",	/*Sale Tax Type 1 Description*/
"TaxType2": 4,	/*Sale Tax Type 2 Internal Identifier*/
"TaxType2Description": "Normal",	/*Sale Tax Type 2 Description*/
"Tax1": 2,	/*Tax 1 Rate Internal Identifier*/


```

"Tax1Id": "13",
"Tax1Rate": 13.0,
"Tax2": 1,
"Tax2Id": "23",
"Tax2Rate": 23.0,
"NetPrice1": 2.92035,
"NetPrice2": 3.79646,
"NetPrice3": 3.89381,
"NetPrice4": 4.0885,
"NetPrice5": 3.60177
/*Tax 1 Rate Code*/
/*Tax 1 Rate Percent*/
/*Tax 2 Rate Internal Identifier*/
/*Tax 2 Rate Code*/
/*Tax 2 Rate Percent*/
/*Net Sale Price 1 - Without VAT included*/
/*Net Sale Price 2 - Without VAT included*/
/*Net Sale Price 3 - Without VAT included*/
/*Net Sale Price 4 - Without VAT included*/
/*Net Sale Price 5 - Without VAT included*/

```

Promotion Menus Items

With this Web Service you can access the full registered Promotion Menus information and their Levels composition.

[https://\[environment\].storesace.com/dimas/api/pos/menuitems/](https://[environment].storesace.com/dimas/api/pos/menuitems/)

Important Considerations:

- Everytime a new Menu, Level or Level Item is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```

[{
  "Id": 1,
  "MenuItemId": "1234",
  "MenuItemDescription": "Menu Almoço",
  "Level": 1,
  "LevelDescription": "Nível 1",
  "Mandatory": true,
  "LevelItemId": "12760",
  "LevelItemDescription": "Sopa Dia",
  "LevelFamilyId": null,
  "LevelFamilyLevel": null,
  "LevelFamilyLevelId": null,
  "LevelFamilyDescription": null,
  "LevelFamilyFullPath": null,
  "ExtraPrice": 0.0,
  "PriceLineId": null,
  "IsOffer": false,
  "Status": 1,
  /*StoresAce Unique Menu Level Item Integer Identifier*/
  /*Item Code for the Menu Item*/
  /*Item Description for the Menu Item*/
  /*Level Index*/
  /*Level Name*/
  /*Mandatory Item Selection Level*/
  /*Level Item Code*/
  /*Level Item Name*/
  /*Level Item Family Code*/
  /*Level Item Family Level*/
  /*Level Item Family Level Code*/
  /*Level Item Family Name*/
  /*Level Item Family Full Names Path*/
  /*Extra Price to apply in Menu*/
  /*Reference Sale Price to apply (1 to 5)*/
  /*True if is to be considered offer, false if not*/
  /*Status: 1 - Active, 0 - Inactive*/
}

```

```
"synccounter": 1615379496,           /*Level Item EPOCH Synccounter*/  
"Updatestamp": "2016-03-10T12:31:36", /*Last Updated Date and Time*/  
"Createstamp": null,                 /*Created Date and Time*/  
"deleted": 0,                         /*Deleted Status: 1 - Deleted, 0 - Not Deleted*/  
}}
```

Customers

With this Web Service you can access the full registered Customers information.

[https://\[environment\].storesace.com/dimas/api/pos/customers/](https://[environment].storesace.com/dimas/api/pos/customers/)

Check the [Customers](#) definition described in HeadOffice methods for more information about this service.

Employees (POS Salesman)

With this Web Service you can access the full registered Employees information.

[https://\[environment\].storesace.com/dimas/api/pos/salemsans/](https://[environment].storesace.com/dimas/api/pos/salemsans/)

Check the [Employees](#) definition described in HeadOffice methods for more information about this service.

Payment Methods

With this Web Service you can access the full registered Employees information.

[https://\[environment\].storesace.com/dimas/api/pos/paymenttypes/](https://[environment].storesace.com/dimas/api/pos/paymenttypes/)

Check the [Payment Methods](#) definition described in HeadOffice methods for more information about this service.

Sales Zones (Locals)

With this Web Service you can access the full registered Sales Zones information.

[https://\[environment\].storesace.com/dimas/api/pos/zones/](https://[environment].storesace.com/dimas/api/pos/zones/)

Important Considerations:

- Everytime a new Zone is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```
[{
  "Id": 1,                                /*StoresAce Unique Integer Identifier*/
  "ZoneId": 1,                            /*Zone Code*/
  "ZoneDescription": "Zona 1",            /*Zone Name*/
  "ZoneShortDescription": "Zona 1",       /*Zone Short Name*/
  "TableRange": "100,103",               /*Table Range*/
  "ZoneType": 0,                          /*Zone Type*/
  "SalesPriceIndex": 1,                  /*Sales Price Index to apply (1 to 5)*/
  "Status": 1,                           /*Status: 1 if Active, 0 if Inactive*/
  "Inactive": false,                     /*true if Inactive, false if Active*/
  "deleted": 0,                           /*Deleted Status: 1 if Deleted, 0 if not Deleted*/
  "synccounter": 1562689257,              /*Zone EPOCH Synccounter*/
  "Updatestamp": "2016-08-05T14:32:46",   /*Last Updated Date and Time*/
  "Createstamp": "2016-07-09T16:20:58"    /*Created Date and Time*/
}]
```

Cash Drawers (Tills)

With this Web Service you can access the full registered Cash Drawers information.

[https://\[environment\].storesace.com/dimas/api/pos/cashdrawers/](https://[environment].storesace.com/dimas/api/pos/cashdrawers/)

Important Considerations:

- Everytime a new Cash Drawer is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

The Output in Json format is as follows:

```
{
  "Id": 1,                                /*StoresAce Unique Integer Identifier*/
  "TillId": "1",                           /*Cash drawer Unique Code*/
  "Description": "Caixa 1",                 /*Cash drawer Name*/
}
```

```

"ShortDescription": "Caixa 1",
"MinimumWarningAmount": 0.0,
"DefaultCashBalanceAmount": 0.0,
"MaximumWarningAmount": 0.0,
"MaximumLockoutAmount": 0.0,
"TillOpenMode": null,
"OnTouchUseRoute": false,
"CanHaveOpenTill": false,
"TillLockToSalesman": false,
"FixSalesmanForSession": false,
"CanChangeTranOnSignOffTill": false,
"CheckForTempTransactionOnSignOff": false,
"OnStatementCanHaveDiscrepancy": false,
"StatementDiscrepancyAmount": 0.0,
"OperatorTenderStatement": false,
"ExclusiveToOwnerSalesman": false,
"TenderStatementTentatives": 0,
"Comments": null,
"DiscrepancyTillId": null,
"DiscrepancyAccountType": null,
"OfflineTill": false,
"MainTill": false,
"MaximumSessionsPerDay": 1,
"IsOpen": 0,
"PattyCash": null,
"Status": 1,
"Inactive": false,
"StatusCode": 1,
"deleted": 0,
"synccounter": 1562689257,
"Updatestamp": "2016-08-05T14:32:46",
"Createstamp": "2016-07-09T16:20:58"
}
/*Cash drawer Short Name*/
/*Minimum Balance*/
/*Default Cash Balance Value*/
/*Maximum Balance*/
/*Lock Balance*/
/*Open Mode configuration (Depends on the POS)*/
/*Ask for Salesman route*/
/*Allows multiple open sessions*/
/*Can only be changed by open salesman*/
/*Set document to current session salesman*/
/*Can change transaction after Cash close*/
/*Check existence of temporary transactions on close*/
/*Allows Cash differences*/
/*Cash close differency allowance*/
/*Blind Cash Close (Salesman don't see Theoric values)*/
/*Can only be closed by main salesman*/
/*Max number of tender amount definition*/
/*Comments*/
/*Move differences to another till*/
/*For negative differences move to another till*/
/*Is this a offline till*/
/*Is this the Main till*/
/*Maximum number of sessions per day*/
/*Open Status: 1 if Open, 0 if Closed*/
/*Amount referenced when opening the Cash Drawer*/
/*Status: 1 if Active, 0 if Inactive*/
/*true if Inactive, false if Active*/
/*Status: 1 if Active, 0 if Inactive*/
/*Deleted Status: 1 if Deleted, 0 if not Deleted*/
/*Cash Drawer EPOCH Synccounter*/
/*Last Updated Date and Time*/
/*Created Date and Time*/

```

Cash Drawers Movements (Tender Transactions)

With this Web Service you can access the full registered Sales Zones information.

[https://\[environment\].storesace.com/dimas/api/pos/zones/](https://[environment].storesace.com/dimas/api/pos/zones/)

Important Considerations:

- Everytime a new Zone is created or an existing one is modified and then marked to Synchronize, it gains a new Synccounter value with current EPOCH date and time.

Lisboa
Av. Eng. Arantes e Oliveira, 3 R/C, Lisboa, Portugal

Porto / Head Office
Rua Fernando Namora 117, Porto, Portugal

info@storesace.com

For the ones with multi-stores
Anyplace, anytime and anywhere.

Level up!
storesace.com

The Output in Json format is as follows:

```
{
  "Id": 123456,                                /*StoresAce Unique Integer Identifier*/
  "TillId": "8",                               /*Cash drawer code for current movement*/
  "TransDocument": "FCX",                     /*Cash drawer Movement Type: FCX Cash Drawer Close*/
  "TenderTransactionNo": "FCX A08/665",        /*Cash drawer Movement Full Document Code*/
  "TenderTransactionUID": "A08|FCX|665",       /*Cash drawer Movement Transaction Identifier*/
  "TransSerial": "A08",                       /*Cash drawer Movement Serial*/
  "TransDocNumber": 665,                      /*Cash drawer Movement Number*/
  "CreateDate": "2023-03-01",                 /*Created Date*/
  "CreateTime": "2023-03-01T00:01:53.426000", /*Created Date and Time*/
  "SalesmanId": "603",                       /*Cash drawer Movement Salesman Code*/
  "TotalAmount": 3102.37,                    /*Total Amount value*/
  "StoreId": "1",                            /*Store Code*/
  "StoreName": "Lisboa",                     /*Store Name*/
  "PattyCash": 60.0,                         /*Amount referenced when opening the Cash Drawer*/
  "IncludesPattyCash": 1,                    /*Includes opening amount value*/
  "synccounter": 0,                          /*Movement EPOCH Synccounter*/
  "Updatestamp": "2023-03-01T00:01:54.489264", /*Last Updated Date and Time*/
  "Createstamp": "2023-03-01T00:01:54.489250", /*Created Date and Time*/
  "TenderTransactionDetails": [               /*List of Tenders amount declaration*/
    {
      "Id": 32785,                            /*StoresAce Unique Tender Movement Integer Identifier*/
      "TransSerial": "A08",                   /*Cash drawer Movement Serial*/
      "TransDocNumber": 665,                  /*Cash drawer Movement Number*/
      "TenderId": 1,                          /*Payment Method Tender Code*/
      "Amount": 194.64,                       /*Tender Amount declared*/
      "TenderDescription": "Numerário",       /*Tender Description*/
      "TransDocument": "FCX"                 /*Cash drawer Movement Type: FCX Cash Drawer Close*/
    },
    {
      "Id": 32786,                            /*StoresAce Unique Tender Movement Integer Identifier*/
      "TransSerial": "A08",                   /*Cash drawer Movement Serial*/
      "TransDocNumber": 665,                  /*Cash drawer Movement Number*/
      "TenderId": 2,                          /*Payment Method Tender Code*/
      "Amount": 1976.9,                       /*Tender Amount declared*/
      "TenderDescription": "Multibanco",      /*Tender Name*/
      "TransDocument": "FCX"                 /*Cash drawer Movement Type: FCX Cash Drawer Close*/
    },
    {
      "Id": 32787,                            /*StoresAce Unique Tender Movement Integer Identifier*/
      "TransSerial": "A08",                   /*Cash drawer Movement Serial*/
      "TransDocNumber": 665,                  /*Cash drawer Movement Number*/
      "TenderId": 3,                          /*Payment Method Tender Code*/
      "Amount": 930.83,                       /*Tender Amount declared*/
      "TenderDescription": "Visa",            /*Tender Name*/
    }
  ]
}
```

```
} "TransDocument": "FCX" /*Cash drawer Movement Type: FCX Cash Drawer Close*/  
}  
}
```

Loyalty API

The Loyalty API Root contains all methods used to integrate with loyalty and promotions systems.

It has the capability to identify the customer purchase details, evaluate the offers available to redeem and return the list of valid offers, vouchers and discounts that can be applied to current Customer Transaction.

The Loyalty API Root is available at:

[https://\[environment\].storesace.com/dimas/api/loyalty/](https://[environment].storesace.com/dimas/api/loyalty/)

Check Available Offers

Use this Service to pass to StoresAce the customer identification (Loyalty Card Number) and the current active Sale Details. StoresAce will validate and output all available offers, discounts and vouchers valid to redeem on current transaction.

The POST data in Json format should be as follows:

```
{
  "store": 1,
  "datetime": "2017-07-14T11:09:56"
  "customer_count": 1,
  "table_total": 489,
  "number_of_items_in_table": 2,
  "customer_card_id": "911234567",
  "document_id": "123456",
  "sale_details": [
    {
      "item": 6947,
      "quantity": 1,
      "item_unit_price": 199,
      "discount_value": 0,
      "sale_value": 199,
      "is_offer": 0
    }
  ]
}
```

/*Store Internal Identifier Id*/
/*Sale Transaction Date and Time*/,
/*Number of customers in sale (Not Required)*/
/*Transaction Total multiplied by 100 (without decimals)*/
/*Number of items details in Sale (Not Required)*/
/*Customer Identification Loyalty Card/Mobile Number*/
/*Number of the Transaction Document*/
/*List of Sale Details*/

/*Item Internal Id*/
/*Quantity*/
/*Item Sale Price with VAT multiplied by 100*/
/*Discount with VAT multiplied by 100*/
/*Total Item Sale Price with VAT multiplied by 100*/
/* Just a Check 1/0 to indicate if it was an offer or not*/

The Check Method API response in Json format is as follows:

```
{
  "customer":{
    "card_id":"911234567",
    "name":"João",
    "customer_name":"João",
    "tax_number":null,
    "email_address":"",
    "points":10,
    "balance":0
  },
  "points":{
    "id":14,
    "name":"1 Icecream = 1 Point",
    "current":10,
    "credit":1,
    "total":11,
    "printer_header_text":"",
    "printer_footer_text":""
  },
  "items":[
    {
      "id":15,
      "name":"10 Points = 1 Ice Cream",
      "item_id":6947,
      "item_code":"900438",
      "item_name":"Magnum Brownie",
      "items_list":[
        {
          "item_id":6947,
          "item_code":"900438",
          "item_name":"Magnum Brownie"
        }
      ],
      "quantity":1,
      "value":0,
      "percent":0,
      "debit_points":10,
      "printer_header_text":"",
      "printer_footer_text":""
    }
  ]
}
```

/*Customer Information do Display in Screen*/
 /*Loyalty Card Number*/
 /*Customer Short Name*/
 /*Customer Short Name*/
 /*Customer Tax Number*/
 /*Email*/
 /*Current Points*/
 /*Current Balance Amount*/
 /*This has the Campaigns that will credit points to the customer for this transaction*/
 /*Campaign Id - This will be required to Send to the Rescue the Campaign ID selected to apply*/
 /*Campaign Name*/
 /*Current Points*/
 /*Number of Points to Credit*/
 /*Final points after the Credit*/
 /*Extra Information to Print in Ticket Header (Paper Ticket)*/
 /*Extra Information to Print in Ticket Footer (Paper Ticket)*/
 /*This are the Campaigns available to the customer to apply discounts/offers*/
 /*Campaign ID - This will be required to Send to the [Rescue](#) the Campaign ID selected to apply*/
 /*Campaign Name*/
 /*Main Item Internal Identifier Id*/
 /*Main Item Code*/
 /*Main Item Name*/
 /*List of Items available to offer/apply discount (when multiple items may exist)*/
 /*Item Internal Identifier Id*/
 /*Item Code*/
 /*Item Name*/
 /*Quantity to Offer*/
 /*Discount Value to apply*/
 /*Discount Percent to Apply*/
 /*Entra info on Points that will be removed to the customer when selecting this campaign*/
 /*Extra Information to Print in Ticket Header (Paper Ticket)*/
 /*Extra Information to Print in Ticket Footer (Paper Ticket)*/

Rescue Offer

Call this Service after Sale Transaction is Closed on Point of Sale, and pass to StoresAce the selected rescue offer from Check Method and also the final Sale and Prices details.

The POST data in Json format should be as follows:

```
{
  "store": 1,
  "datetime": "2017-07-14T12:44",
  "customer_count": 1,
  "table_total": 250,
  "number_of_items_in_table": 1,
  "customer_card_id": "911234567",
  "document_id": "FS 970226396",
  "sale_details": [
    {
      "item": 6947,
      "quantity": 1,
      "item_unit_price": 199,
      "discount_value": 199,
      "sale_value": 0,
      "is_offer": 1
    }
  ],
  "rescued_campaigns": [ 15 ]
}
```

/*Store Internal Identifier Id*/
 /*Sale Transaction Date and Time*/,
 /*Number of customers in sale (Not Required)*/
 /*Transaction Total multiplied by 100 (without decimals)*/
 /*Number of items details in Sale (Not Required)*/
 /*Customer Identification Loyalty Card/Mobile Number*/
 /*Final Sale Transaction Document Number*/
 /*List of Sale Details*/
 /*Item Internal Id*/
 /*Quantity*/
 /*Item Sale Price with VAT multiplied by 100*/
 /*Discount with VAT multiplied by 100*/
 /*Total Item Sale Price with VAT multiplied by 100*/
 /*Just a Check 1/0 to indicate if it was an offer or not*/
 /*LIST of check items campaigns IDs selected to redeem*/

Customers Purchases API

The Customers Purchases API Root contains all methods used to integrate purchases from several external sources.

The Customers Purchases Root is available at:

[https://\[environment\].storesace.com/dimas/api/customerspurchases](https://[environment].storesace.com/dimas/api/customerspurchases)

Purchases

Use this Service to create or update (status) a given purchase of a customer.

The POST data in Json format should be as follows:

```
{
  "store_id": 1,                /*Store Internal Identifier Id*/
  "origin": 3,                  /*Fixed value 3 */
  "doc_status": 3,              /* Status 3 (Paid), 7(Pending Delivery),
                                8 (Delivered), 9( Returned) */

  "ref_doc_number": "123",      /*Purchase ID */
  "ref_doc_date": "2021-03-15", /*Purchase Date*/
  "obs": "Comment",             /* Comments Optional*/

  "customer_name": "João",      /* Customer Name*/
  "customer_tax_number": "",     /* Customer Tax Number*/
  "customer_email": "",         /* Customer Email*/
  "customer_telephone": "912333444", /* Customer Telephone*/

  "shipping_method": 1,         /*1 (Home Delivery) , 3 (Store Pickup)*/
  "shipping_agent": "DPD",       /* Description of Shipping Agent Optional*/
  "shipping_customer_name": "João", /* Shipping Customer Name Optional*/
  "shipping_tax_number": "",      /* Shipping Customer Tax Number Optional*/
  "shipping_phone": "912333444",  /* Shipping Customer Telephone Optional*/
  "shipping_address": "Rua Porto", /* Shipping Address Optional*/
  "shipping_postal_code": "4000-000", /* Shipping Postal Code Optional*/
  "shipping_country": "PT",       /* Shipping Country Optional*/
  "shipping_tracking_code": "PT00134", /* Shipping Tracking Code Optional*/
  "shipping_costs": 0             /* Shipping Costs Optional*/
}
```

```

"billing_customer_name": "João",
"billing_tax_number": "",
"billing_phone": "912333444",
"billing_address": "Rua Porto",
"billing_postal_code": "4000-000",
"billing_country": "PT",

"total_net": 25.33,
"total_vat": 5.83,
"total_doc": 31.16,
"accounting_discount": 0,
"CustomersPurchasesHeadersDetails": [{
    "item_id": 100000,
    "quantity": 1.0,
    "unit_price": 16.99,
    "unit_discount": 5.1,
    "total_net": 9.667,
    "total_price": 11.89
}],
"CustomersPurchasesHeadersTenders": [{
    "payment_type_code": 2,
    "payment_description": "MB",
    "value": 31.16,
    "entity": null,
    "reference": null
}]
}

```

```

/* Billing Customer Name */
/* Billing Customer Tax Number */
/* Billing Customer Telephone */
/* Billing Customer Address */
/* Billing Customer Postal Code */
/* Billing Customer Country */

/* Purchase Total without Vat */
/* Purchase Total Vat */
/* Purchase Total */
/* Purchase Global Discount Optional */

/* Line ItemId /pos/itemsfull/ */
/* Line Quantity */
/* Line Unit Price */
/* Line Unit Discount */
/* Line Total without Vat */
/* Line Total */

/* Block Optional */
/* TenderId /pos/paymenttypes/ */
/* Payment Description */
/* Value */
/* MBRef Entity Optional */
/* MBRef Reference Optional */

```

To get Customer Purchases use the following GET services

Purchases for the day

[https://\[environment\].storesace.com/dimas/api/customerspurchases/?format=json&store=1&date=2023-11-17](https://[environment].storesace.com/dimas/api/customerspurchases/?format=json&store=1&date=2023-11-17)

Purchases for Customer

[https://\[environment\].storesace.com/dimas/api/customerspurchases/?format=json&store=1&customerid=111](https://[environment].storesace.com/dimas/api/customerspurchases/?format=json&store=1&customerid=111)

Purchases for Customer for given day

[https://\[environment\].storesace.com/dimas/api/customerspurchases/?format=json&store=1&date=2023-11-17&customerid=111](https://[environment].storesace.com/dimas/api/customerspurchases/?format=json&store=1&date=2023-11-17&customerid=111)

Payload for these GET are similar to the POST described above

WebHooks

There are two webhooks available:

first when the order is processed it is possible to send the generated invoice (pdf in base64) and AT code, so it's possible to inform the logistic distributor;

second In the case of a store pick-up order, it is possible to inform when the customer collected the order, this way it is possible to update the customer's profile .

Help & Support

For any clarification of the information present in this document, please contact:

Email:	api@storesace.com
--------	--